



M363 Unit 1
UNDERGRADUATE COMPUTING

Software engineering with objects



Approaches to software
development

Unit **1**

This publication forms part of an Open University course M363 *Software engineering with objects*. Details of this and other Open University courses can be obtained from the Student Registration and Enquiry Service, The Open University, PO Box 197, Milton Keynes MK7 6BJ, United Kingdom: tel. +44 (0)845 300 60 90, email general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by The Open University.

To purchase a selection of Open University course materials visit <http://www.ouw.co.uk>, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858793; fax +44 (0)1908 858787; email ouw-customer-services@open.ac.uk

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc; Motif is a registered trademark of the Open Group; Rational Unified Process is a registered trademark of International Business Machines Corporation; Unified Modeling Language and UML are trademarks of Object Management Group, Inc. Design by Contract is a trademark of Interactive Software Engineering. Other product and company names may appear in the M363 course material. Rather than use a trademark symbol with every occurrence of a trademarked name, we use the names only in an editorial fashion and to the benefit of the trademark owner, with no intention of infringement of the trademark.

The Open University
Walton Hall
Milton Keynes
MK7 6AA

First published 2008.

Copyright © 2008 The Open University.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS; website <http://www.cla.co.uk>

Open University course materials may also be made available in electronic formats for use by students of the University. All rights, including copyright and related rights and database rights, in electronic course materials and their contents are owned by or licensed to The Open University, or otherwise used by The Open University as permitted by applicable law.

In using electronic course materials and their contents you agree that your use will be solely for the purposes of following an Open University course of study or otherwise as licensed by The Open University or its assigns.

Except as permitted above you undertake not to copy, store in any medium (including electronic storage or use in a website), distribute, transmit or retransmit, broadcast, modify or show in public such electronic materials in whole or in part without the prior written consent of The Open University or in accordance with the Copyright, Designs and Patents Act 1988.

Edited and designed by The Open University.

Typeset by The Open University.

Printed in the United Kingdom by Thanet Press Ltd, Margate

ISBN 978 0 7492 1607 8



CONTENTS

1	Introduction	5
2	Software and software engineering	6
	2.1 What is a system?	6
	2.2 The nature of software	8
	2.3 Characteristics of a software system	8
	2.4 Maintainability and other software problems	10
	2.5 Divide and conquer?	12
	2.6 Architecture and components	19
	2.7 Summary of section	21
3	An introduction to software development	22
	3.1 Software development as an engineering activity	22
	3.2 The role of development processes	23
	3.3 Software systems come in all shapes and sizes	28
	3.4 Traceability	33
	3.5 Emergent approaches to software development	35
	3.6 Summary of section	38
4	Modelling	39
	4.1 From methods to modelling	39
	4.2 Models illustrate points of view	42
	4.3 Summary of section	45
5	Activities and artefacts	46
	5.1 Domain modelling	46
	5.2 Requirements engineering	48
	5.3 Further analysis	48
	5.4 Design	49
	5.5 Implementation	49
	5.6 Testing	49
	5.7 Deployment models	50
6	CASE tools	51
	6.1 Benefits and costs	51
	6.2 A taxonomy of tools	52
	6.3 Summary of section	53
7	Summary	54
	References	56
	Index	57

M363 COURSE TEAM

Robin Laney, Course Chair, Author and Academic Editor

Leonor Barroca, Author

Ralph Greenwell, Course Manager

Charles Haley, Author

Nigel Kermode, Critical Reader

Martin Shepperd, External Assessor, Brunel University

Richard Walker, Critical Reader

Andy Allum, Course Website Developer

Tara Marshall, Print Procurement

Kim Dulson, Software Procurement

Phillip Howe, Media Assistant

Callum Lester, Software Developer

Sandy Nicholson, Freelance Copy Editor

Andy Seddon, Media Project Manager

Lucinda Simpson, Editor

Sue Stavert, Technical Tester

Andrew Whitehead, Graphic Artist

Thanks are due to the Desktop Publishing Unit, Faculty of Mathematics and Computing.

1

Introduction

In this course we present an **engineering** approach to the development of software systems. This means we pay particular attention to issues of **software quality** – both in terms of product (what is built) and process (how we build it).

The material in the course is organised in a distinctive way. Object-oriented software development uses a fairly standard set of development techniques. In M363 we take a broad view, and these techniques are discussed fairly independently of exactly where and when they would be used. In parallel with this, however, *Units 4, 8, and 12* feature a running case study that demonstrates how the techniques might be applied in practice.

There has always been debate in the computing industry about just how useful any particular development technique is. We believe that the techniques that have been introduced over the last 30 years or so, and many others currently being researched, are important because they address fundamental issues concerning software quality. Studying these techniques is as much about coming to an understanding of the quality issues as it is about the nuts and bolts of the techniques themselves.

With experience you may find that some issues can be dealt with in a more lightweight manner than the one we explain, but our goal is for you to have a thorough understanding of what the issues are, and for you to be able to address them in at least one way.

Software systems are built to meet requirements. A successful software project must:

- ▶ resolve the diverse and possibly conflicting needs of users in a disciplined way;
- ▶ satisfy the users' expectations;
- ▶ have been developed and delivered in a timely and economical manner;
- ▶ be resilient to the changes that will be introduced during its operational lifetime.

This is a daunting prospect for those developing and maintaining the software. A central aim of this course is to give you the intellectual tools to cope with the challenge.

This unit provides an introduction to software engineering. Assuming you already have some experience of software development, some of this material will be familiar to you, though your existing knowledge will be consolidated and will begin to be extended to more advanced areas. We discuss some of the ideas that underpin software development in Section 2, and the basic activities of software development are considered in Section 3. In Section 4, we consider the role of models and modelling languages. Section 5 presents the specific way in which we will organise the development process for the main case study. Finally, Section 6 deals with **CASE tools** – computer-aided software engineering tools – designed to support the professional developer.

This leads to the developer's mantra, 'Software must be delivered on time, to budget and to specification'. You may know it in another form.

2

Software and software engineering

This section describes the basic characteristics of a 'good software system', and considers how such software systems may be developed. Once built, software rarely remains static and can change on a regular basis. Therefore, maintaining software is a key activity in software engineering. Our intention in this section is to provide the background for a more detailed discussion that comes later. Therefore, we recommend that you read quickly through this section and then reread it once you have gained some more insight into the software development process, both at the end of this unit and again at the end of the course.

2.1 What is a system?

You might consult a dictionary and find that system is derived from the Greek meaning 'to set up'.

The word *system* is in regular everyday use. We talk about the social-security system, the telephone system, computing systems and even 'The System'. The trouble is that our everyday speech is often imprecise, and people use the same word to mean different things (which is one reason why software development can be an arduous process!).

We define a **system** as 'an assembly of components that are connected together in an organised way'. The term *organised* is very important here. For example, it tells us that components are affected by being part of a system, and that they change if they leave the system.

Example 1

A pile of old telephones, exchange-switching gear and wiring that are awaiting disposal may once have been a system. But heaped together, they are *not* a system. What has been lost is the *organisation* and the *connections*.

A system, therefore, is greater than the sum of its parts, and it has properties that cannot be deduced or predicted by examining any one of its components in isolation (or even *all* its components, if they are not organised and connected). From the above example, could you deduce the properties of the telephone system by examining a single, disconnected telephone (or even a big pile of disconnected telephones)? No, because it no longer works as expected, even though it has not changed physically (it is not broken).

The assembly of components that forms a system does something, which is to say that systems are dynamic in some way, and processes of some sort or another go on within them. If you view your body as a system, it consists of organs and does something, which is live, even when you are asleep or in some comatose state. While you are asleep, blood circulation, respiration and metabolism continue. We use the concept of a **process** to provide support for separate activities, so that we can deal with the amount of work to be done to support blood circulation and respiration, for example. Systems can contain concurrency.

A system can be, and often is, a personal ordering of reality, the result of seeing some degree of orderly interconnectedness in some part of the world. Thus a system can be many different kinds of system simultaneously, depending on who is studying it and why. In this sense, it is a subjective ordering of reality.

Example 2

A telephone system is a communications system to its users. For the engineers who set it up and maintain it, it is a technical system (which is linked to an employment system, a *job*, in their view). Similarly, someone who designs telephone switches would consider each switch as a system in its own right. A switch can also record usage data besides routing your calls.

As Example 2 shows, there is a notion of what is included within a system, and what is excluded. We would like to identify the scope of a system. The **system boundary** is simply a conceptual line that divides the system that we wish to study from ‘everything else’. It is useful to think of a system’s *environment* as being made up of those things which are not part of the system, but which can either affect the system or be affected by it. Example 3 takes you into a particular area of interest, which is known as a **domain**, to look at system boundaries and how they can change.

Example 3

Hospitals are a domain where you can find software being put to a variety of uses. A hospital might, for example, join together a series of patient-monitoring systems along with the database management system that manages medical records, to make a larger system with a different scope. A forward-looking hospital might also wish to go further and add weather-forecasting software. The extension would allow planners to deal with the ebb and flow of patients that arise according to the season. Beds may be allocated, and other resources, such as drugs, can be bought in preparation.

SAQ 1

In Example 3, we suggested that two systems, for patient monitoring and managing medical records, might be combined into a single system. Suggest an additional function that might be possible with the combined system that would not have been possible with either of the two original systems alone. What can you say about the boundary of the combined system compared with the boundaries of the original systems?

ANSWER.....

Suppose the monitoring system detected that a patient taking a common drug had a heart problem. If the two systems were combined, it would be possible to automatically check whether the heart problem might be due to a known allergy recorded in the patient’s record.

The boundary of the combined system encompasses a wider scope than the combined boundaries of the separate systems because of their relationship.

2.2 The nature of software

Software is often spoken of as being ‘invisible’ or ‘intangible’, and hence is thought of as being very different from physical artefacts, which can be measured, touched, broken, and so on. This ‘invisibility’ can lead to unrealistic expectations concerning the capabilities of software. These expectations may well contribute to some of the myths that surround software and its development.

There are three important characteristics of software that affect its development.

- ▶ **Malleability.** Software is easy to change (all programmers are often tempted to ‘tweak’ their code). This malleability creates a constant pressure for software to be changed rather than replaced. Every change that is made to the software introduces the possibility of new errors.
- ▶ **Complexity.** Software is often complex. Complexity can usually be recognised, but it is less easy to define. One item of software can be considered more complex than another if the description of the first requires more explanation than that of the second. Part of that complexity arises from the potential variety of pathways between the components of a system. The number of errors is likely to depend on the complexity of a system.
- ▶ **Size.** It is likely that there will be more errors in a large piece of software than there will be in a small one.

You have already encountered an aspect of the ‘intangible’ nature of software if you have programmed in, for example, Java. The instructions and statements that you write in Java are translated into bytecode, which you do not see.

You might think that, because it is ‘invisible’, software is inherently more difficult to develop than a physical artefact. However various techniques can be used to model software and its behaviour (just as an industrial designer uses geometric abstractions and other tools to model a physical product before it is built). We will explore such uses of models throughout the course.

SAQ 2

For each of the three characteristics of software mentioned above, explain how errors might arise in a piece of developed software.

ANSWER.....

Malleability: every time a change is made to a piece of software it introduces the possibility of a new error; the easier it is to make changes, the greater the chances of introducing new errors into the software.

Complexity: the more complex a piece of software becomes, the more chances there are of introducing an error.

Size: there are likely to be more errors in a large piece of software than in a small one.

2.3 Characteristics of a software system

Having considered the basic terms *system* and *software*, we can move on to the notion of a software system. For the purposes of this course, a **software system** is equivalent to applications software. There are two important questions that we want to address.

- ▶ What characteristics should we be looking for in a software system so that we can develop one that meets the needs of all its users?
- ▶ What attributes should a software system have in order to determine its quality?

Ultimately, we develop a software system in response to an identified need to solve a customer's problem. The contract between customer and developer, as provider, includes the requirements specification, which identifies what the software should do and the environment in which it must work. We expect a software system to be *useful*, as otherwise it cannot meet the needs of its users.

To its users, the user interface represents the software system. No matter what functionality is contained in the software, it cannot be used to its full potential if that interface is difficult to use. We expect a software system to be *usable*; otherwise the users' tasks are hindered.

Software can and does contain errors. We expect a software system to be *reliable*, in that errors are minimised, as otherwise its users will not be able to perform those tasks that need its support.

While the developers are busy constructing a software system, it is probable that the users' needs will change. Once that software system becomes operational, new requirements are expected. It is also possible that the developer may miss a requirement during the specification process. We expect a software system to be *flexible*, since it is important to be able to change it easily as time goes by. In addition, a flexible software system makes it easy to correct errors.

In order to operate in its target environment, any design solution to a requirements specification must be turned into machine-readable code. The accepted method of doing this is to represent a design using a programming language, and then use an interpreter or compiler to generate machine-readable code. No matter how useful or usable a software system might be, we expect it to be *available* in the target environment.

There will also be a contract between the customer and the developer that includes delivery dates and costs. Whatever process is used, the developer is expected to meet such contractual constraints. We could also say that the developer's working practices affect the availability of a software system, which includes its initial delivery and any subsequent changes to it. From the customer's point of view, the software system must be *affordable* to buy and maintain.

From the developer's point of view, there must be a way of keeping control of a project so that the software system is affordable. Labour resources are the most significant component of a developer's contract, and often become the two-edged sword that leads to the success or failure of a software development project. We will return to this aspect of software development later in the course, when we deal with professional issues.

Much of this course is concerned with the activities that lead to a useful software system – techniques that help us define a requirements specification and then produce a design solution to the problem contained in the specification.

SAQ 3

- (a) What is the defining quality of a good software system, and what are its main characteristics?
- (b) How might greater flexibility make a software system more affordable?
- (c) Give two reasons why a delivered software system might not meet its users' needs.

ANSWER.....

- (a) A good software system is one that meets its users' needs. We can characterise a good software system as useful, usable, reliable, flexible, affordable and available.
- (b) Users' needs will change over time. The time taken to implement the changes in requirements is less than if that software were inflexible. Since labour costs are the most significant component of software costs, flexible software is more affordable.
- (c) Software systems are usually out of date even as they are being developed, since users' needs:
 - ▶ are often missed during requirements capture;
 - ▶ change with time.

2.4 Maintainability and other software problems

You will study maintainability and other quality factors in Unit 11.

A software system should be both available, so that users can decide whether or not it still meets their needs, and flexible, so that the developer can change it to meet its users' needs. As we will show in Section 3 of this unit, *maintenance* is the key activity for the coordination and control of changes to a software system. In order to be maintainable, a software system should be written and documented in such a way that changes can readily be made. This means that we must take into account the process used to develop a software system. What the developer does during development affects the ease with which it can be maintained. If a change is easy to make, the cost of that change (labour) can be minimised so that the software system continues to be affordable. The **maintainability** of software is greatly influenced by how it is designed and written.

Example 4

For anything you buy and use, there are costs associated with its purchase and with the maintenance required for its continued operation. For example, there are costs associated with buying a car and with its continued operation. As the vehicle ages, the costs of maintenance rise, and there comes a point when you have to decide whether it would be preferable to buy a new car or continue with the existing one. Surprisingly, perhaps, there are similarities between the maintenance of vehicles and the maintenance of software.

A significant problem relates to software systems that have been in operation for some time. If a particular software system continues to meet its users' needs, there may be little or no motivation to change or replace it, especially if that software is associated with a critical service within a company. Such systems are called **legacy systems**. We typically find they:

- ▶ are large;
- ▶ are critical to the business;
- ▶ have been changed a number of times since their inception;
- ▶ are difficult to understand because of either a lack of documentation about their internal structure or a lack of experience within the group responsible for them;
- ▶ are difficult to maintain because of the above factors.

One option would be to replace an ageing software system with a new one. But the change from old to new can have serious implications for the company involved. It is not just the users who need to be retrained to use the new software. In some companies the whole internal organisation is based around their major software systems. Change these and the whole company may be faced with a costly reorganisation to reflect the new software system more accurately. There is also the issue of maintaining continuity of service during the changeover, and the risk that the new and therefore unknown system may not work. The legacy system is a known quantity.

The same problems also apply to software that is purchased in an 'off-the-shelf' package, which has been an increasing trend since the 1990s.

There is an additional twist that makes the problem of legacy systems worse. Staff turnover may mean that there is no one left in the company who understands the legacy system enough to continue maintaining it. Even if those who developed the original software system have not left, they may well be working on other projects or be unwilling to look after an 'old system'. When faced with the choice of working on a 'new system' or an 'old system', many people prefer to produce something new.

There is every chance that tomorrow's legacy systems are being built today. We might say that today's solutions are tomorrow's problems.

Example 5

In 1992, the London Ambulance Service commissioned a computer-aided dispatch (CAD) system for getting ambulances and their crews to reported incidents. It was intended to replace an existing paper-based system. There were two notable failures of the new system in November 1992 that resulted in severe delays in ambulances reaching certain incidents. The final report on the system failures (Page, 1993) identified a number of major management problems with the project. A technical audit revealed that the CAD system was incomplete and not fully tested. Its ability to deal with heavy loads had not been tested; neither had a backup service been tested. There were also outstanding problems with the accuracy of the information needed to initiate each dispatch.

Unfortunately, successful software systems rarely make the news. Example 5 illustrates the kind of failure that does break through into the public domain (in this case, the failure became a series of headline stories). The London Ambulance Service had a software system that was lacking in usefulness, usability, reliability and availability. In the long term, it may not be considered affordable because of its performance in the target environment.

This discussion of software problems has probably given you a rather pessimistic view of software development. However, you must not forget that many software systems work, and they work very well, given the fact that they are used in every aspect of modern life. However, the rate at which the role of software is evolving is so great that there must be practices involved in developing software that can cope effectively with this expansion.

SAQ 4

- (a) Suggest a means of measuring the maintainability of a software system.
- (b) How does a legacy system relate to the attributes of a good software system?
- (c) Suggest a reason why legacy systems will always be a problem.

ANSWER.....

- (a) We could measure the effort required by a developer to locate and eventually implement a given change to a software system. We could classify that effort in two components: the effort needed to locate and fix errors (bugs); and the effort needed to adapt the software system to meet its users' needs.
- (b) A legacy system is lacking in flexibility. It may have started out with all the characteristics of a good software system, yet those characteristics may have changed over time. The longer a software system is used, the more difficult it will become to maintain, because of the number of changes made since its development. So it gradually loses its maintainability. As a software system ages, the quality of applied changes will be affected by their affordability and the relative importance of the system. Ultimately all the staff who developed and knew about the software system may have moved on, so those new staff allocated to its maintenance may not fully comprehend its features. This can happen to all software systems, including those being developed today.

Thus a legacy system still serves some useful purpose, but there are problems that make it less flexible and maintainable (and so affordable) than it was originally.

- (c) The inherent malleability of software makes it easy to change. You have already seen that a legacy system is lacking in flexibility as a result of the number of changes made to it during its operational lifetime. (The analogy with metal-working through malleability is useful. Once a blacksmith forms some component, usually in iron, there is a limit to the number of times that it can be heated, formed and cooled before that component becomes brittle and hence liable to failure.)

This explains why our ability to bolt features and fixes onto a legacy system means that it will eventually become too fragile, and it will become precarious to go any further. The staff issues mentioned in (b) compound these problems.

2.5

Divide and conquer?

As computing technology has improved, we have tried to construct software systems that can deal with larger and more complex problems. In order to provide such solutions, the software systems themselves have become larger and more complex. Unfortunately there is a limit to how much we can take in and understand at any one time. How can we cope with 'tricky' problems or situations where there is just too much information?

The main technique for dealing with such messy situations is **decomposition**. We can decompose a problem into smaller and smaller parts or chunks until each one can be comprehended or dealt with by an individual. In terms of our earlier definition of a system, we are looking for patterns and/or components within a system and creating internal boundaries around them to identify smaller subsystems, sometimes referred to as **modules**. We will expose some form of hierarchy in our attempt to simplify a complex system.

The concept of decomposition can also be applied to the way we develop a software system. As you will see in Section 3, we can identify a number of different activities or tasks that an individual or group of individuals might perform in a software development project.

The Romans had a strategy called *divide et impera* – 'divide and rule'. However, this covered the idea that it was easier to rule over groups at conflict with each other.

An individual can successfully build small software systems because it is possible for that person to understand all that is needed about the problem and its solution. There is a long history of individuals attempting to develop more complex systems – so-called **heroic programming** – where success has been less certain. However, there are systems that are so large and complex, such as those that monitor and control air traffic, that they cannot be built by a single individual. The development of such a software system requires a team of people whose work must be well coordinated and managed; there must be a well defined process if they are to produce an appropriate solution, that is, a software system that is useful, usable, reliable, flexible, available and affordable. In practice, the process of software development is partitioned to enable individuals to specialise in different development activities such as analysis, design and implementation.

Problem and solution

By dividing one large problem into a set of smaller subproblems, we might expect to reach a point where we can capture, understand and describe each subproblem. But there are two difficulties.

- ▶ How do we know that each identified subproblem is any easier to understand and solve than the original problem?
- ▶ How do we know that all of our subproblems will fit back together again and recreate that original problem?

The key to finding subproblems that you recognise and understand is prior knowledge and experience. As a developer, you may know how to create a good design or a good program, but you may not be familiar with the **problem domain**.

As a developer, the problem that you face is in the world outside the computing system, where you aim to provide a solution. The software that we construct can provide solutions to problems, because they are connected to the world outside, as illustrated in Figure 1.

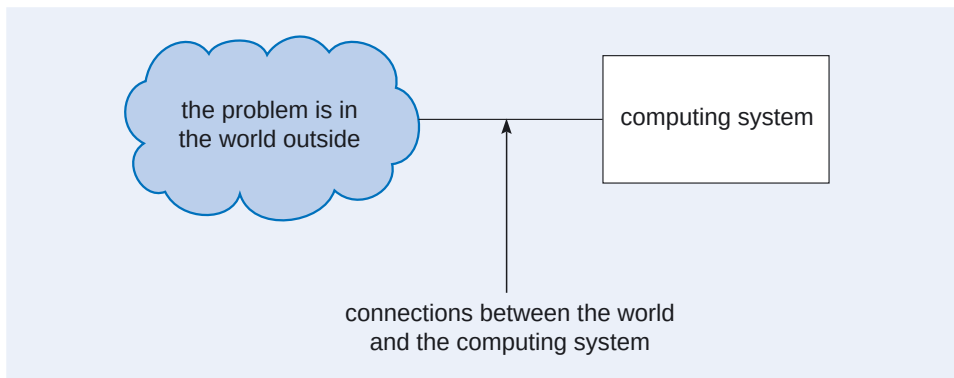


Figure 1 Separating a problem from its solution

The main issue that you, as a developer, should focus on is solving the problems identified by the users, rather than worrying about how to write the program code. When projects fail it is nearly always because of mistakes that were made quite early on in the design stage, not because of what happens at the coding stage.

Example 6

One way to find out whether you have a good solution to a problem is to consider the kinds of complaint you would receive from users if your software did not work properly. For example, suppose someone had made a reservation to stay in a hotel but when they arrived, the receptionist told them that there

was no reservation in their name. In another case, they might have reserved rooms for six people only to find that there was only room for one. By doing such thought experiments we may often be able to spot flaws in our solution, although of course there may always be situations we have overlooked, so we can never be completely certain that our solution is good!

Projections and partitions

In an ideal world, we would be able to decompose any problem into a number of mutually independent subproblems, known as **partitions**. In practice, we usually find that there are common elements shared between subproblems. Instead of producing partitions, we find that our decompositions lead to projections. Figure 2 illustrates the difference between the two. Since projections overlap, there will be elements in the area that are found in more than one projection. For example, in Figure 2, you may find the same element in projections B, C and D. In a partitioned area, each element appears in exactly one partition. For example, an element inside partition E is not found in partitions F, G, H or J.

It is easier to comprehend partitions than projections. Because partitions are independent, we can concentrate upon each one in turn, ignoring the others. Hence we favour methods of decomposition that allow us to minimise the amount of overlap between subproblems.

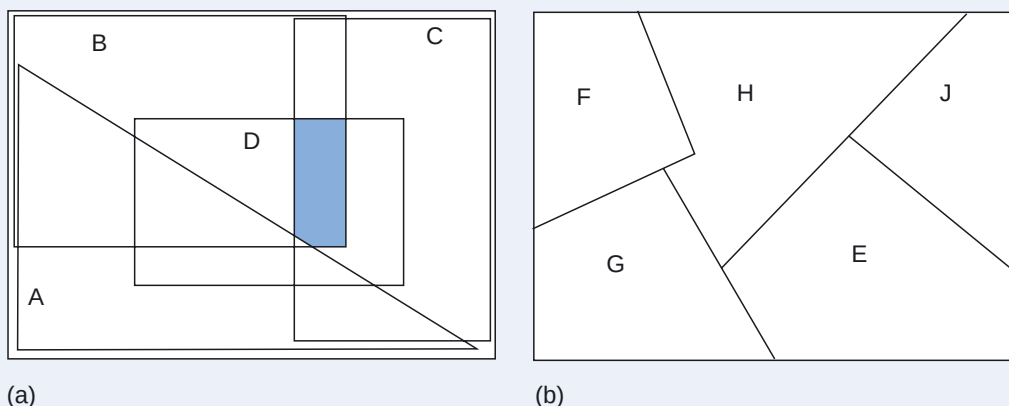


Figure 2 Decomposition by (a) projection and (b) partition

In software development, there is a long history of decomposing a system into separately named and addressable components or elements, called *modules*. The modules partition the system, for example, its design or code, while the overall design of the system must take into account any projections in the problem space. Examples of modules are:

- ▶ whole programs or applications;
- ▶ software libraries;
- ▶ classes, in an object-oriented language such as Java.

Although modules may appear as independent elements, there are relationships between them that need to be taken into account, as Example 7 shows. However no other general technique exists for coping with large and complex problems, so in practice we use **modularisation**, and accept that there will be a degree of interdependence between modules.

Example 7

In developing a software system for a typical manufacturing business, we can identify a number of possible subsystems, that is, areas for development, such as systems to deal with customers, production, accounts (payments) and deliveries. We might want to treat these areas as independent (that is, as partitions). However, there are connections between them; for example, customers are expected to pay for the goods that they have ordered from the business, and the deliveries department is expected to deliver to the customer. Similarly in the production subsystem, the progress of each customer's order would be tracked through the factory and on to the delivery of the completed goods to the customer.

Coupling

As Example 7 shows, developers need to deal with the dependencies that arise as a result of their decomposition of a problem and its solution into a number of components or modules. We say that a component of a system depends on another if it is possible that a change to one component requires a change to another. For example, if a business changes its production methods this may cause a consequent change in the way it calculates the payments required for the goods it produces.

A developer must not only deal with the nature of each dependency but also the number of dependencies. In software engineering, the term **coupling** is used to refer to the degree of interdependence among the different components of a system. It is easy to see that certain systems can have chains of interdependent modules where, for example, module A depends on module B, which depends on module C, and so on. In some cases these chains may join up and create a **circular dependency**, which is a particular form of strong (or high) coupling.

Developers try to construct loosely coupled systems, since they are easier to understand and modify. So a good software system has low coupling, which means that changes to one component or module are less likely to propagate through the rest of the system. A further benefit of low coupling is that components are easy to replace and, potentially, reuse.

In an object-oriented software system, two classes need to be coupled in order to allow any communication between their instances. We say that coupling exists between two classes if one class can make a direct reference to the other. The class A is coupled to the class B if any of the following applies:

- ▶ A inherits from B;
- ▶ A has an attribute of class B;
- ▶ A has a method that uses an instance of class B as an input or output argument;
- ▶ A knows of a public attribute of class B.

Whatever the level of coupling in a software system, it is important to know which modules (or classes) are coupled. If there were no record of the coupling between modules, a developer would have to spend time working through the modules to determine whether or not a module was affected by a change. The result would be a lot of effort spent on checking, even if no changes were needed. Example 8 illustrates the danger of having more than one module making use of a common or shared area.

One solution was developed for relational databases. There is a standard way to deal with dates, which is defined as part of the Structured Query Language (SQL).

Example 8

Date handling has always been a problem for software developers. For applications of a certain age, the most applicable storage format for representing a year was a number between 0 and 99. It made sense: 1966 was stored as 66, 1989 as 89, and so on; less space was needed to store just two digits. Furthermore, if dates were stored as numbers, tasks that involved sorting by date order were simple: 22 January 1989, stored as 890122, is after 22 December 1966, stored as 661222. Unfortunately, a number of these applications were still in use as 2000 approached; so every module in every application that used the short form of year had to be investigated.

A major aspect of the problem in Example 8 was that different developers had different ways of reading and manipulating the values stored in variables that used the six-figure date format. This increased the effort required to resolve the 'millennium bug', as it became known. If there had been a consistent way for developers to manipulate dates that did not rely upon the storage format, the impact of the millennium bug would have been practically non-existent.

Interfaces

Sometimes, you can define more than one interface for a module to allow yourself to be more precise about which services will be offered to different kinds of client.

In general, we say that the **interface** to a module defines how other modules can use that module. An interface is the means of connecting one module to another. It tells you what to expect about the behaviour of a given module and what services the module will provide without telling you how those services will be provided. For example, the interface can define how a date module will respond to queries about today's date or the number of days between two dates. For a class called `Date`, its interface is a description of all the externally visible operations without any details of how those operations are implemented. From an object-oriented point of view, we say that the interface to the class `Date` is an **encapsulation** of what we know about `Date`.

A module, which provides services to other modules, may in turn need to use the services of yet other modules. These required services are called its **context dependencies**. We can describe the quality of service given by a particular module by its ability to gain access to the services that it needs in order to work. Taken together, a module's interface and its context dependencies can be read as a contract. Each contract describes what the module does and what it needs to do it. Potential clients of that module can assume that it will undertake certain responsibilities under certain specified conditions.

The concept of an interface helps the developer to be more productive. If you can rely on the specification contained in a module's interface, then there is less for you to understand because you do not need to know how it works. Furthermore, you have a better chance of understanding your part of the software system because you can focus on the things you need to perform your task. A side effect of this added understanding is that you are less likely to introduce errors. Furthermore, if a module is hidden behind an interface it is potentially replaceable.

Once your software system contains a set of modules that are well understood, each with its own interface and context dependencies, you can consider whether any of them can be reused. It may be that a popular set of modules is adopted as a standard. The ability for different computing systems to communicate over a network has developed because of the adoption of such a standard set of modules. Control over the complexity of computer communications has been gained by decomposing the problem into a number of layers. Each layer has a well defined interface through which the layer above it accesses its services.

Cohesion

Cohesion is a way of describing how closely the activities within a single module are related to each other. Cohesion is a general concept; for example, a department in an organisation might have a cohesive set of responsibilities (accounts, say), or not (miscellaneous services).

A highly cohesive module performs one task or achieves a single objective: 'do one thing and do it well' is a useful motto to apply. A module should implement a single logical task or a single logical entity. From an object-oriented point of view, a class can meet both of these requirements provided it contains methods appropriate to one kind of object.

Abstraction is a particular way of viewing a complex problem to arrive at some useful decomposition of that problem. The idea is to group together similar objects or situations and, while ignoring their differences, focus on one particular aspect of the problem. The key task in developing a software system is to decide upon the most suitable abstractions in the problem domain.

We can say that we have achieved a useful abstraction in a particular module if the potential software clients of that module do not need to know more than is contained in its interface. For example, a dedicated module to deal with date handling is a useful abstraction. The fact that it may be complex to implement is immaterial to clients that use the services defined in its interface.

Low coupling and high cohesion are competing goals. If every module does only one thing at a low level of abstraction, we might need a complex edifice of highly coupled modules to perform an activity at higher levels of abstraction. A developer should try to achieve the best balance between the levels of coupling and cohesion for a software system. For example, hotels generate income by letting out their rooms to guests. The concept of room is likely to be represented somewhere in the software system for reservations for a hotel. It may be convenient to use a module or class representing the concept of room to collect and store data about the income generated by letting rooms. However, a better solution is to have a separate bill or payment module, because it is more cohesive, especially when a hotel generates income in other ways, for example, from its meal service provided to people who are not resident guests.

SAQ 5

- (a) Why might you consider splitting up a large project into smaller chunks?
- (b) How does the complexity of a software system affect the maintenance task?
- (c) What is a module?
- (d) Why does it help to have low coupling in a software system?
- (e) Give examples of the kinds of information that would be valuable when considering a change to a given module.
- (f) What are the context dependencies of a module? How do they relate to a module's interface?
- (g) What are the benefits of using modules with defined interfaces?
- (h) Why does it help to have high cohesion in the modules of a software system?
- (i) What characteristics should a module display that will help to ensure that it is easy and cheap to develop and maintain, and that errors are kept to a minimum?
- (j) Why is it important to achieve a balance between coupling and cohesion?

ANSWER.....

- (a) There is a limit to how much a person can understand at any one time. So there is a limit to the size of a software system that any one person can deal with. By splitting a large project into smaller chunks, it is possible to identify a number of more manageable tasks for those involved.
- (b) It is essential to be able to make a change to a software system without having to know all about that system. Each change becomes difficult when the flow of control and dependencies within programs are complex. The greater the number and nature of the dependencies, the harder it is to maintain a software system.
- (c) A module is any identifiable part of a software system that is considered separately. For example, modules may be subroutines (in a procedural language equivalent to methods), classes (in an object-oriented language), library functions or other constructs that may be treated independently.
- (d) With low coupling, there are few dependencies between modules. Therefore changes made to one part (one or more modules) of a software system are less likely to propagate throughout the whole system. (A clear record of the dependencies between modules helps you to predict the impact of a proposed change to a software system.)
- (e) There are two kinds of information that contribute to the analysis of a proposed change, and these are as follows.
 - ▶ What assumptions have been made in other modules (clients) that call upon the module in question? An understanding of the expected services of a module will help assess the risks associated with a particular change.
 - ▶ Which modules are clients of the module in question? This information indicates how far a change may propagate through the software system.
- (f) The context dependencies for a module are those services that the module needs in order to work correctly. You can express the context dependencies for a module in terms of other interfaces. In effect, you can express the responsibilities of a module in terms of its interface and context dependencies. If the context provides the services that the module needs, then the module can guarantee the provision of the services described in its interface.
- (g) The benefits are as follows:
 - ▶ developers will need to know only about the module's interface (its syntax and what it achieves – its semantics), not how it provides those services, and consequently developers can be more productive;
 - ▶ developers can understand aspects of the software system more thoroughly, so fewer bugs will be introduced;
 - ▶ it should be easier to find bugs, as irrelevant modules are avoided;
 - ▶ the possibility of module reuse is increased once it is known what that module provides and requires.
- (h) With high cohesion, a module carries out a sensible set of operations or activities; ideally, high cohesion implies just one major abstraction per module. The interface abstracts away from what a developer must know in order to use a module. The developer can concentrate on the work required and create software that is useful, usable, reliable and affordable. Indeed, high cohesion tends to make a module more usable.
- (i) A module should have low coupling and high cohesion, a good abstraction, and a well defined interface that is an encapsulated abstraction of a well understood concept.

- (j) In constructing a system, we may have a choice between a smaller set of loosely coupled, less cohesive modules, or a larger set of tightly coupled, more cohesive modules. In the former case each module may be difficult to understand; in the latter case the relationships between them may be over-complex. We need to strike an appropriate balance.

2.6 Architecture and components

The term **architecture**, in software development, is associated with the overall structure of a software system or, at a higher level, a family of software systems. For this course, we will adopt the following definition, taken from Bass, Clements and Kazman (2003).

The **software architecture** of a program or computing system is the structure or structures of the system, which comprise software elements, the externally visible properties of those elements, and the relationships among them.

It is important to understand the software architecture because it tells the developer, among other interested parties, about the overall shape of the actual or proposed software system. It explains how the development team can use the various technologies to construct or assemble a software system.

Some of the material in this section is derived from Cheesman and Daniels (2000).

Choosing a software architecture, or at least part of it, is one of the earliest decisions a development team has to make. There is always something that can be called a software architecture, whatever software system is being developed. Even the heroic programmers choose a basic software architecture when they choose which programming language and operating system to use.

The use of the term *element*, rather than *component*, in the definition of software architecture is helpful. This is because the term *component* is used to mean a variety of things, each one with some validity in its own context. In this course, we will use the term **component** to denote a unit of reuse or replacement in a software system. So, a component could be a module or class with certain properties that make it reusable or replaceable in a given software architecture. What is important is to have one, or possibly more than one, standard in mind when you are deciding whether or not a particular module or class is a component. This is where the software architecture comes in. The larger the family of software systems governed by a software architecture, the greater the potential for reuse and replacement for anything you call a component.

A major aspect of software architecture is the identification of the different partitions into which you can put the various pieces of software that are going to be used to provide your solution to a problem. Figure 3 illustrates one instance of the *N*-layered structures that can be found in distributed computing systems.

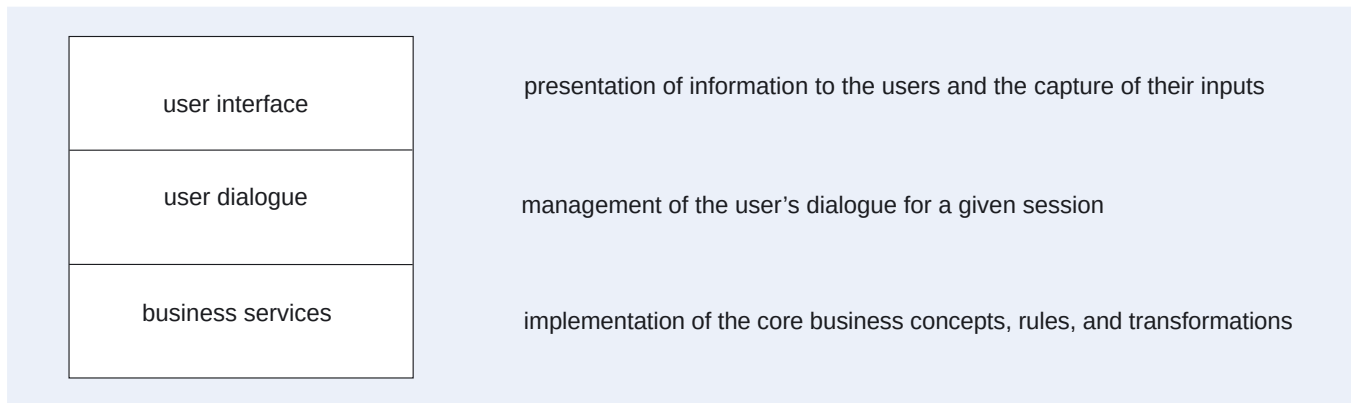


Figure 3 A three-layered architecture

A variety of software architectures is also useful when you need to assess the impact of any proposed change to the software systems that conform to it. You can say that a software architecture identifies a set of rules for decomposition – the assumptions used to modularise a software system. In Figure 3, for example, the user interface concentrates on presentation aspects, which are more prone to change than the rest of a software system. (It is natural to expect a number of requests from users to make a software system more usable.) The user dialogue is concerned with support for the way in which a user performs a given task, such as the processing of a customer's order. A business may redesign the tasks that its employees perform, but not as often as amendments will be made to the user interface. Unless a business makes a radical shift in its core business concepts and transformations, the business services are less prone to change than the user dialogues. In the next section, we consider how you can develop a good software system, as our focus shifts from product to process. As you will see, software requirements come in all shapes and sizes; so developers need to consider what is the most appropriate process for a given context.

SAQ 6

- (a) What are the characteristics of a component?
- (b) How does the concept of an architecture contribute to component reuse?
- (c) Which form of decomposition might be used in a software architecture?

ANSWER.....

- (a) A component is a module that is considered to be a sufficiently good abstraction for the problem in hand. A component should be capable of being reused in future projects having the same software architecture, or being easily replaced at a later date within the existing software system. A good component has a well defined interface and is an encapsulated abstraction of a well understood concept.
- (b) The architecture of a (software) system embodies high-level decisions about the overall structure of the system, and this architecture may apply to more than one system. (An approach called *product-line development*, covered later in M363, will produce a sequence of systems that have much in common, through a shared architecture, and makes software more affordable if it uses the same components.)
- (c) Complexity is best managed by decomposing a problem into smaller sub-problems. The basic form of decomposition used in a software architecture is partitioning to meet a number of separate concerns. For example, we might want to separate the user interface from the core business services.

Exercise 1



- (a) Why would a good software system need to be both useful and usable?
- (b) If a piece of software contained bugs, how might they affect the usefulness and usability of that software?

Solution

- (a) If a software system is not useful, it will not meet the users' needs; if it is not usable, the users will be hindered from performing the tasks they need the software for.
 - (b) If a piece of software contained bugs, it might not meet the users' needs at all in some respect, or it might meet their needs but in a way that affected usability and made it harder for users to carry out particular tasks.
-

2.7 Summary of section

This section has briefly examined the nature of software, and identified the desirable characteristics of a software system. You have seen:

- ▶ that a good software system is one that meets its users' needs;
- ▶ examples that illustrated the connections between the usefulness, usability, reliability, flexibility, availability and affordability of a software system;
- ▶ that a software system can soon be out of date, as users' needs change with time, and that needs can often be missed during requirements capture;
- ▶ that modularisation is the main method of dealing with the size and complexity of a (potential) software system;
- ▶ the problems that arise with legacy systems;
- ▶ the significance of maintenance.

Software systems are becoming all-pervasive in our society, and the demand for new systems is growing rapidly. There are significant risks associated with software systems when they are critical to a business. This all points to the need for software development processes that will deliver software systems that are easy to maintain and reliable, while at the same time ensuring that the systems serve some useful purpose for their users. Good development processes will produce well engineered components based on reusable architectures.

3

An introduction to software development

In this section we will introduce the basic activities involved in the development of software. Then, we will consider the general concept of a life cycle for software development and consider examples of different life cycles. At the same time, we will consider the importance of models as part of software development.

3.1 Software development as an engineering activity

Software development has a great deal in common with the discipline of engineering, from which the term software engineering arose, and is said by Dr. Friedrich L. Bauer, one of the founders of computer science, to be:

The establishment and use of sound engineering principles in order to obtain economically, software that is reliable and works efficiently on real machines.

(Bauer, in Pressman, 2001)

Carrying out a process 'economically' implies management, and we assume that **project management** is a necessary activity within the discipline of software development.

Words such as 'reliable', 'efficient', 'flexible' and 'maintainable' (among others) describe characteristics of software systems. They are all aspects of *software quality*. To obtain a high-quality software product requires a well managed development process.

But what leads us to use the term *engineering* for software development?

Our response to this question includes the following observations.

- 1 Developers are concerned with meeting a set of requirements; there is an identifiable problem that they try to solve.
- 2 There is a defined process that can be used to produce a solution and, within each process, there are a number of identifiable phases or activities.
- 3 There are tasks to be done in each phase that result in one, or possibly more than one, product (software, hardware or document). Developers undertake different roles to perform such tasks (designer, programmer, tester and so on).
- 4 The quality of both the products and the processes by which they are made is important: validation, verification and testing in general play an important role in ensuring high quality.
- 5 There are tools to improve the effectiveness and efficiency of the tasks performed by developers in their various roles.
- 6 There is a body of knowledge that developers might use and/or add to. There are ways of working to support the other five observations (such as standards and rules for decomposition in a given context).

You might like to look at some dictionary definitions of the term 'engineering' and see how the above observations fit them.

While there is considerable argument about the term *software engineering*, there is no doubt that software development does require certain skills and knowledge of certain ideas

and principles. This section looks at what is needed to ‘engineer software’. What should a developer do in order to produce a software system that meets the needs of its users?

In Section 2, you saw that there are three characteristics of software that affect its development and use: malleability, complexity and size. We said that the number of errors in a system increases with complexity and size, and whenever a system is changed there is a risk of introducing new errors. Therefore, we adopt the principle of modularisation in order to cope with the issues of size and complexity. We need to pay special attention to the development process to ensure that the introduction of errors is minimised whenever a change occurs.

Software development is a human activity. Software systems are developed by one group of people for another group of people to use. The success or failure of a project is determined by social factors as well as technical factors.

Users want a system that does what they need and that they can use. One which is technically superior but does not meet these criteria will fail. Users will resent, ignore or, in the worst case, completely reject it.

SAQ 7

Give the characteristics of an engineering approach that support the argument that software development is an engineering discipline.

ANSWER.....

Software development follows an engineering approach provided that those involved:

- ▶ are concerned with meeting a clear set of requirements that are defined as clearly as possible;
- ▶ use a defined process with clear activities, each of which has at least one identifiable end-product;
- ▶ can apply their skills and experience to the tasks demanded of them;
- ▶ regard validation and verification to be as essential as building the software itself;
- ▶ make sensible use of tools and standards.

3.2 The role of development processes

A development process is a set of rules that define how a software development project should be carried out. The identification of a set of software engineering activities associated with the development of software is the key to meeting the potential diversity of users’ needs. It is the primary mechanism for understanding and harmonising these sometimes conflicting demands on the development process. Each activity will undertake some clearly defined process, starting with a number of inputs from any preceding activities that are the basis for further work. On completion of an activity there may be one or more outputs, which are known as **deliverables**. The order in which these activities are carried out is called a **life cycle**, and outlines an overall process for the development of a software system.

Through such activities we will emphasise a disciplined approach to software development. This has three key elements:

- ▶ **methods**, which describe the technical activities required to develop software;
- ▶ **tools**, which allow computer support for the methods;
- ▶ **procedures**, which provide the glue that holds the methods and tools together; it includes project and quality management activities.

Project management and quality management activities permeate the life cycle of software development. They are an integral part of every software development project, no matter which particular life cycle is used.

In Section 2, we introduced the notion that a good software system is one that meets its users' needs. In general, we will identify a **customer** as someone who pays for a software system, in contrast to the people who use it on a day-to-day basis – the **users** (although a customer may also be one of the intended users). A successful development project will deliver a product that meets, or even exceeds, the customer's expectations in terms of a 'good software system'.

A complete life cycle would take us from the first ideas about the need for a software system to its final withdrawal. We will assume that a customer has assessed the feasibility of some initial ideas with a potential group of users and has decided that a software system *may be* required.

Typical technical activities for the development of software

Space does not permit us to describe the many and varied methods used to develop software. However, each one normally includes activities which can be roughly classified as follows.

- ▶ **Analysis.** As you saw with the definition of a system, you need to be able to include all that you consider to be relevant. This includes **requirements elicitation**, where you identify the problem and then produce a (clear) statement of the problem, and **requirements specification**, where you decide what needs to be done to solve the problem.
- ▶ **Design.** You determine how you will solve the problem.
- ▶ **Implementation.** You act upon the decisions made at the design stage.
- ▶ **Testing.** You apply what you have done so that you can determine whether or not you have solved the problem.

Different methods may subdivide the above activities or use different terminology. A specific approach, the Unified Process (UP), will be illustrated in the main case study, and in Section 5 you will learn more about the way these activities are taught in this course. By themselves, the four activities of analysis, design, implementation and testing are not enough to develop a good software system. Other activities are needed to a greater or lesser extent depending on the context, as you will see in this section. For example, you are likely, first, to break up most problems into smaller, more manageable chunks, and then deal with each one separately. If you do this to develop a software system, it will be necessary to bring the chunks together into a unified whole. This process is known as **integration**, and is sometimes identified as a separate activity. Sometimes delivery of the software system is identified separately, especially when there are contractual implications such as payment.

We expect that a software system will change during its operational lifetime. This is the **maintenance** activity, which allows a software system to evolve in order to:

- ▶ correct errors;
- ▶ adapt the software to a changing environment;
- ▶ introduce enhancements required by the customer;
- ▶ improve the software in anticipation of future changes.

For the purposes of this course, we will assume that you will be developing a software system that requires you to break up the problem into smaller, more manageable chunks. As such, integration is also a necessary task within the maintenance activity. Following an engineering analogy, what you take apart during maintenance, to perform some change, you must put back together.

The four activities of analysis, design, implementation and testing are the ‘backbone’ of the practices that turn software development into an engineering discipline. They are the activities you will see most often in diagrams depicting the life cycle of software development. But it is important to recognise that they are not the only activities involved in the process of developing a good software system. Project management and **quality management** are the two additional activities that hold the process of development together – the all-important ‘glue’ for software engineering activities. Maintenance will inevitably involve the activities of analysis, design, implementation and testing, and will itself require managing, as illustrated in Figure 4. The seven technical activities shown are organised to form a life cycle.

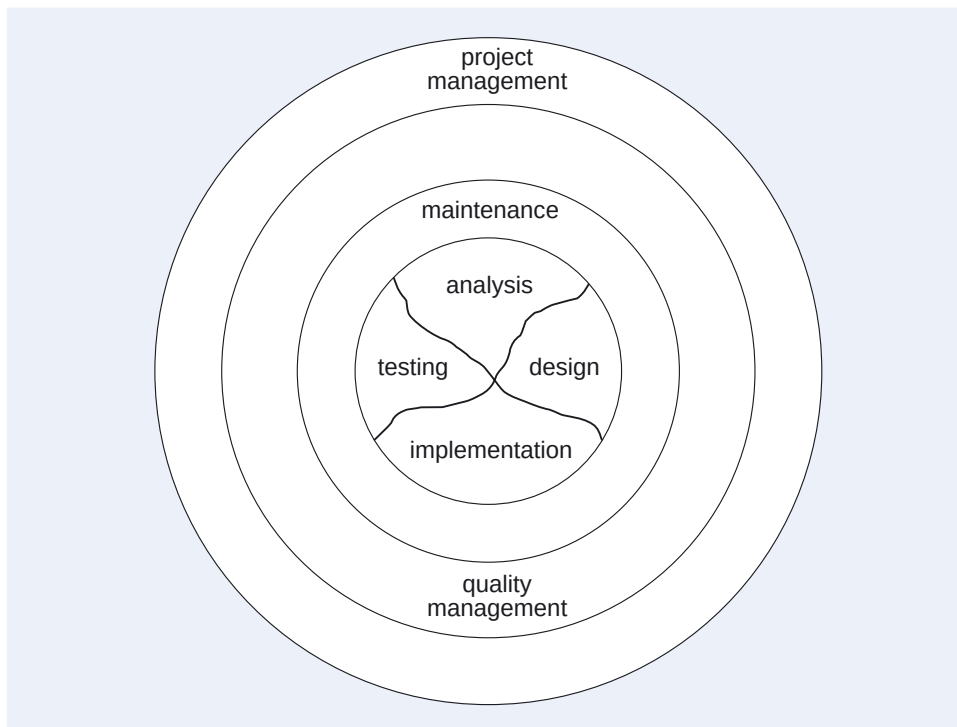


Figure 4 Seven activities for the development of a good software system

An overall process or life cycle

A life cycle is a description of all the events and activities in the life of a software system or product and the sequence in which they happen. Thus, you can choose how to connect the seven activities together to form a life cycle, which you can then use to elaborate a process for developing software. For example, if you arrange the five activities of analysis, design, implementation, testing, and maintenance into a *single sequence*, then you have the classic **waterfall model** for the development of software. However, in practice it is not usually possible to complete each activity correctly in one attempt. In any case, as development proceeds, the products of earlier stages become dated, as our understanding of both the software and its environment evolves.

If projects use a strictly sequential life cycle, a working version of the software system will not be available until late in the testing activity. For larger projects, this could be significant, since it assumes that the customer is patient, and the users are not concerned about the delay required to produce a working response to their requirements. In addition, any errors detected in the working version of the software could be disastrous. Real projects rarely follow a purely sequential life cycle. The act of reviewing is the most important activity when testing the quality of any development process and its resulting products.

An alternative is to iterate around one or more of the activities in a life cycle. **Iteration** allows the developers to improve the outputs from a given set of activities and get feedback before moving on to the next activity. In particular, iteration allows a group of people, usually developers, to perform a review of an activity or the outputs from that activity. In general, reviewing a proposed solution provides the feedback necessary to modify it and improve the solution. Think what happens when you need to tune a guitar or violin. You pluck a string, a note sounds, and you adjust the tension on that string. You repeat the process until you get the desired pitch.

It is often difficult to identify all requirements and state them explicitly at the outset of a development project. The opportunity to iterate around activities such as analysis will increase the chances of successful requirements capture. One way to improve the analysis and identification of requirements is for the developers to produce a **prototype**. A prototype could be a paper-based model, although it is usually a simplified software system that is a subset of the eventual product. It can take several forms:

- ▶ a mock-up of the human–computer interface for a specific group of users (remember that good software systems must be usable);
- ▶ a working version of a subset of the functions required from the eventual product;
- ▶ an existing piece of software that has some relevance to the users' needs and may be considered for further improvement for use in the eventual product.

Figure 5 illustrates a simple prototyping process that can be performed as part of the analysis activity. During requirements analysis, a particular question may be raised that can be answered by the construction and evaluation of a prototype. The customer, in conjunction with the potential users, will evaluate the prototype. There may be some further refinement required before the original question can be resolved satisfactorily, at which point development can continue as usual.

A prototype is not intended to be a complete working version of the software. Therefore it is important that the purpose of each prototype is agreed at the beginning, by both developer and customer, in order to minimise any misunderstanding that may arise. Examining an incomplete system can lead to false impressions and raise expectations among users, but it may be the case that building the prototype leads to a better understanding of other parts of the customer's domain.

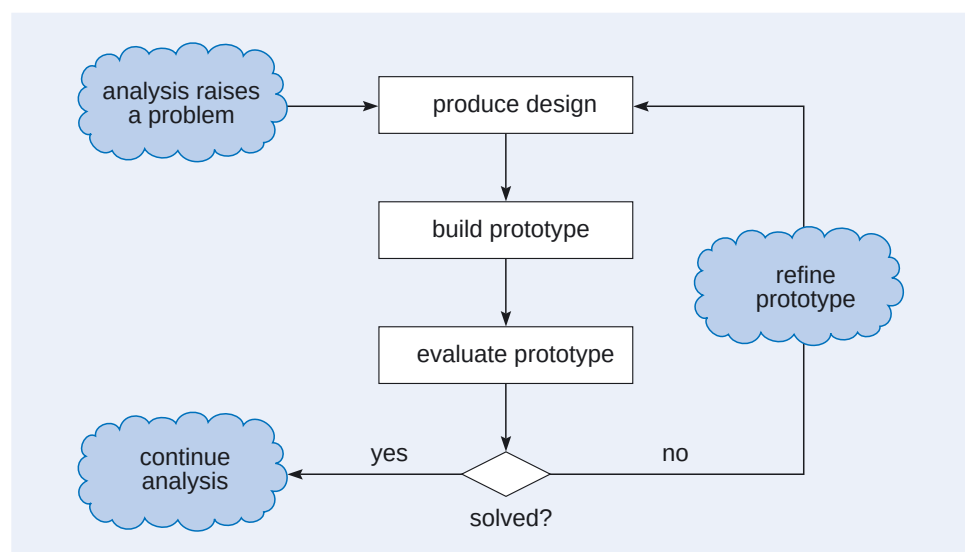


Figure 5 Prototyping during requirements analysis

Incremental development is another form of life cycle. Here the idea is to divide the system into a number of independent or reasonably independent 'chunks'. The process begins by developing one chunk – the core – and adding successive **increments** to it over time. A significant point about this method is that the core is a functioning item of software that meets some of the requirements. As each increment is added, the resulting software is still functional. In this way, the users obtain useful and usable software quickly. This method also enables the developers to take on board feedback from users as the software develops; an increment may simply be an enhancement of the previous version. Increments can be developed sequentially or in parallel, depending on circumstances. For example, a small team might choose to develop increments sequentially, according to a priority agreed with the users.

It is not always possible to partition the users' requirements into independent chunks. So some additional activities will be needed to manage the dependencies between the chunks. On large projects, a single developer may be allocated to this role. Each activity will involve identifying the interfaces that allow communication between one chunk and another. Such activities would coincide with the start of each increment and continue at regular intervals.

Incremental development is a way of dealing with the complexity of requirements, and is analogous to the modularisation discussed in the previous section. Both incremental development and modularisation come from the notion of breaking up a large problem into a number of smaller, more manageable ones. As part of the development process, there will be a number of tasks devoted to putting the pieces back together.

(The activities mentioned here can be assembled into many forms of life cycle. In practice you may meet other approaches to software development, which are outside the scope of this course.)

SAQ 8

- (a) What is a development process?
- (b) What is the role of project management with respect to the deliverables of a development project?
- (c) What is the difference between a customer and a user?
- (d) Suggest a reason why maintenance is a core activity in the development of a good software system.
- (e) What must the users and developers do before developing a prototype, in order to avoid any false expectations?
- (f) What additional task is needed when the development of a software system is partitioned into a number of increments?
- (g) What are the distinguishing characteristics of the waterfall model?
- (h) What is the main function of a prototype?
- (i) What are the problems associated with prototyping?

ANSWER.....

- (a) A development process is a set of rules that define how a software development project should be carried out. It incorporates a life cycle, and involves a number of methods and procedures.
- (b) Each deliverable uses resources, such as developers' time, that add to its cost. A good software system must be affordable, and project management is concerned with controlling the cost of developing such systems. So project management involves the identification of the appropriate deliverables for a given set of requirements, and controlling the cost of producing them. Project management

also involves ensuring that deliverables are produced on time, and taking steps to cope with any delays.

- (c) A customer is the person who pays for a software system, whereas a user is someone who will use that system on a day-to-day basis. A customer will also be a user when the proposed software system is intended to support their job.
- (d) We expect a software system to change during its operational lifetime. Maintenance allows a software system to evolve so that it continues to be useful.
- (e) It is important that both users and developers agree on the purpose of a prototype, since both parties need to avoid any misunderstandings that might arise. For example, users may seek to add new features that undermine the existing agreements. Evaluation of the prototype with respect to its agreed purpose will guard against 'feature creep'.
- (f) A task devoted to the integration of the increments to form the final software system will be needed.
- (g) There is an assumption that once a particular activity or phase is finished, it is not re-entered, and, furthermore, that the activities do not overlap but follow each other in a sequential life cycle. There is no need for reviewing or reworking. All this presupposes that an end point for each activity can be identified.
- (h) The main function of a prototype is to facilitate requirements analysis, and reduce the risk of the requirements being identified incorrectly.
- (i) On the one hand, the client wants to use the prototype as the final system to avoid the delays associated with engineering a proper product. On the other hand, the developer forgets that implementation compromises have been built into the prototype, and begins to treat these compromises as an integral part of the software system.

3.3 Software systems come in all shapes and sizes

A software development method provides an integrated set of techniques for carrying out technical activities in a well defined way. It will tell you:

- ▶ the notation to be used in each activity and the rules for using such notation;
- ▶ the necessary inputs and outputs (deliverables) required for each activity;
- ▶ a process model, that is, how the activities combine to produce the desired result.

The process model provides the framework for the project plan, while the method should allow detailed planning of both the software development process and the quality assurance activities. In principle, a method might be assumed to address every activity and every deliverable product in the life cycle. In practice, this is not always true. Some methods deal with only part of the development process, such as testing – leaving out analysis, design and implementation.

By definition, a good software system *must* be fit for its intended purpose. It should be evident, therefore, that since software is needed for such a variety of purposes, there is no single development process that will suit all purposes. Consider the following examples of requirements:

- ▶ to control a series of gates at level crossings;
- ▶ to control a manufacturing process for chemicals;
- ▶ to manage an international stock market;
- ▶ to manage a supermarket;

- ▶ to manage a public lending library;
- ▶ to administer the activities of a university;
- ▶ to help you manage your personal finances;
- ▶ to control your television and video recorder;
- ▶ to play a game on a mobile phone;
- ▶ to manage electronic funds stored on a smartcard.

The people who use these systems will have different views about what it means to have a software system that is useful, usable, reliable, flexible, available and affordable. So, it should be no surprise that there are different development methods for different types of system. Indeed, software companies often specialise in developing software for specific kinds of business, such as banking or manufacturing.

The following two examples illustrate how the amount of formality in a development process varies.

Example 9

Government legislation often requires software systems to be built to support the new administrative functions. When developers compete to supply such software systems, they are asked to conform to government-approved standards. One of those standards is known as *PRINCE*, which stands for 'PRojects IN Controlled Environments' and relates to the management of projects and the quality of the developed software. Commonly, developers must be able to show that their development process conforms to the regulations set out in the PRINCE method. In addition, developers may be required to adopt an approved method for their analysis and design activities. The most common of these for use with PRINCE is known as the *Structured Software Analysis and Design Method(ology)*, or *SSADM*.

Example 10

In certain financial areas, being first on the market to offer some new service is the way to make the most profit. In other words, the time-to-market is critical: miss out and your competitor takes the spoils. When there is a need for a software system to support such a service, development is almost a race against time. Every aspect of the development process has to be 'tuned' to meet the time-to-market. Thus the aim is to do the simplest things needed that could possibly work when delivered. It also means that communication overheads must be minimised, so there will be very few developers (say, two) who work to a minimal set of agreed procedures. Also, if the amount of actual code produced is small, the chance of introducing errors is reduced. There must still be testing to ensure that the software performs its intended purpose. If the complexity of the software is also minimal, there will be fewer tests to do. But there must still be documentation.

Examples 9 and 10 show that there is no one development process that is appropriate to all kinds of software product. The amount of information recorded during development may be very different in the two examples. However, there is a common need to be able to reconstruct the significant events that happened during the development of the software system, as you will see later.

Software configuration management is the discipline of managing and controlling change in the evolution of software systems (IEEE Std 1042-1987).

Example 9 illustrates the need for a formal development process. Although it is a slow and deliberate course of action, the resultant software system must be able to deal with all the nuances of the legislation.

The developers must be able to show that each aspect of the legislation has been incorporated into the final software system. The control and maintenance of the integrity of a project's deliverables (ensuring that they do not become corrupted or inconsistent) is the responsibility of the **configuration management** activity, which is related to the need for integration described above. In contrast, Example 10 relates to software that is required for a very specialised purpose with a minimal development time. The developers have the benefit of a narrow scope for the software system, and the short development time means that it is unlikely that the users will request many changes, further reducing the demands on development.

In general, the size of a project influences the choice of development process. As illustrated in Example 10, the amount of formality in the process can be minimal. Small teams of up to 10 people can communicate and respond to changes informally. In larger projects, such as in Example 9, a well defined procedure for controlled development is necessary; an informal communication network on its own is not enough. Although the organisational dynamics of team working are beyond the scope of this course, Table 1 illustrates the problem of team size and its effect on communication pathways between the team's members.

Table 1 The effect of team size on communication

Team size	2	4	6	8	10	12	20
Pairwise pathways	1	6	15	28	45	66	190
Potential pathways	2	28	186	1016	5110	24 564	10 485 740

The number of pairwise pathways is $n(n-1)/2$ where n is the team size.

The number of potential pathways is $n(2^{n-1} - 1)$, where n is the team size.

If you count the number of direct paths between all pairs of individuals you get numbers in the second row of Table 1. However, if you consider that messages will be passed on between larger groups, the total number of paths increases dramatically, as shown in the third row of the table. The implication of this is that the more pathways there are, the more time will be taken in ensuring that all members of the team are up to date, and the greater will be the chance of introducing errors.

Figure 6 shows that with 4 people, we could start with anyone – suppose it is the person at the top left – then we have these 7 possible ways that person can communicate with others.

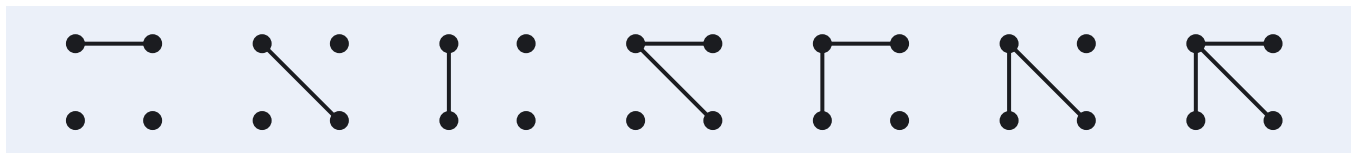


Figure 6 Communication paths

The same pattern applies whoever we start from, hence the total 4×7 , which is 28.

One approach to the solution of the problem of large projects is to split the group into smaller teams according to the responsibilities set out in a given development process. Just as modularisation is the way to deal with the complexity of a software system, developers can be assigned different activities and deliverables within a given project. Naturally, this partitioning allows developers to specialise and become analysts, designers, programmers and so on.

At some point, a software system will have outlived its usefulness. The developers should consider the expected lifetime of the prospective software when they assess the risks of producing a successful product. In Example 10, the time-to-market is a significant factor, but the expected lifetime is likely to be short (because of the nature of financial markets). So a slow and deliberate development process is inappropriate.

Dealing with risk

You have already seen how iteration can be applied to each activity in a life cycle to improve its outputs. The same idea can be applied to a life cycle as a whole. Taking a step back to consider the solution (which will be a software system) in a broader context allows you to manage the risks associated with a development project rather than ignoring them. For example, if delays cause the team in Example 10 to miss the market 'window', it does not matter how much software has been developed, because it is no longer of benefit to the customer and the users. Risk assessment is an important feature of software development.

In a typical project, the major risks are around the requirements. Do you understand them? Have you got them all? Are they changing too frequently? Anything that you can do to increase your confidence that the requirements you have are both necessary and sufficient can reduce the risk of project failure. You have already seen how prototyping can help in requirements analysis. In general, for every decision you make, there is a risk that your decision is wrong, or at least inappropriate, and you should consider identifying and reviewing major decisions. If the risks cannot be overcome, then the project is unlikely to succeed.

In any project there is likely to be a trade-off between what can be delivered in a given time and to a specified budget. Often, the number of desirable features of a solution exceeds those that can be delivered for a given price and within a given time scale; so choices have to be made. One way to make such choices is to estimate the risk of not delivering each feature. That is, if a feature were not to appear in the final product, what effect would this have on such things as the usability, usefulness, reliability and flexibility of the product?

Figure 7 illustrates a simple spiral process that can be used in the development of a software system. There are four steps that are repeated with each iteration of the spiral:

- ▶ determine the objectives, the alternatives and the constraints;
- ▶ evaluate the objectives; identify and resolve the associated risks;
- ▶ develop and verify a (partial) solution or product;
- ▶ review that solution, and plan the activities for the next iteration.

'One pass, one solution' is just the same as the waterfall model.

There are many interpretations and variations upon Barry Boehm's original spiral.

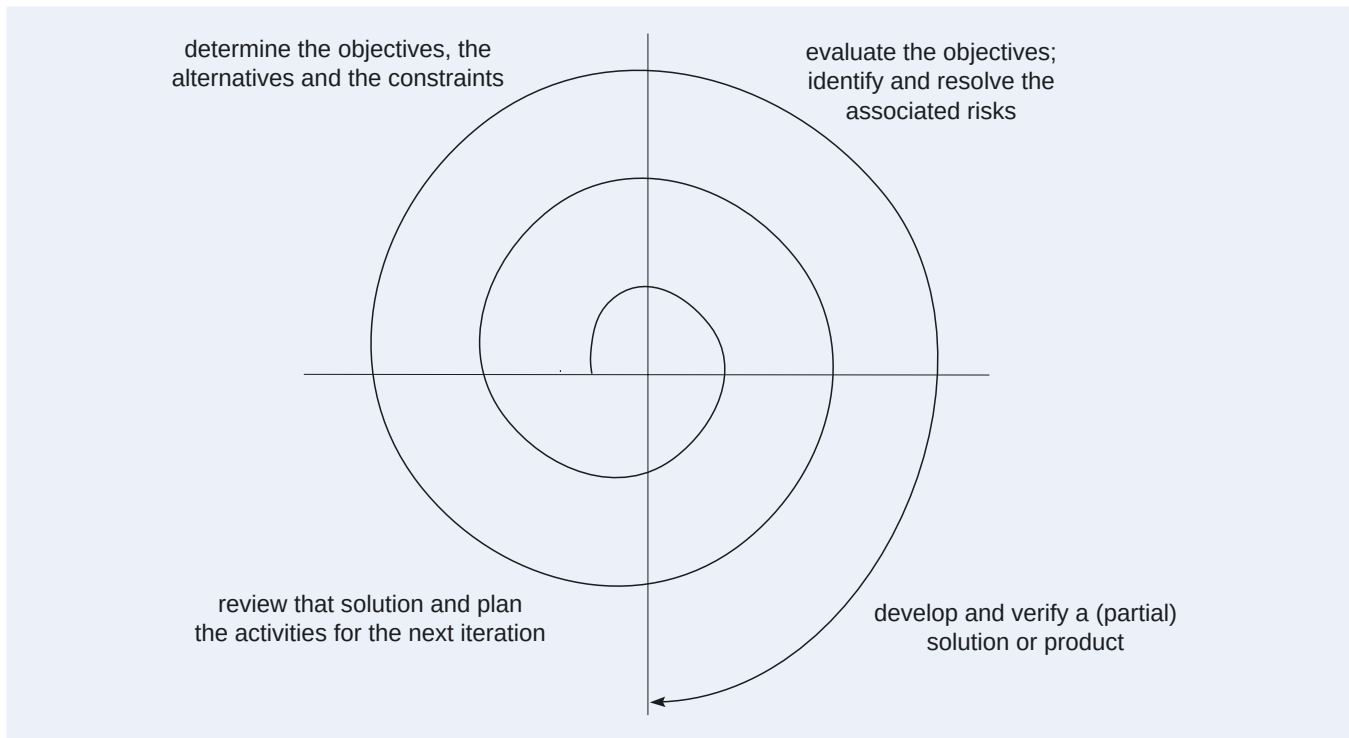


Figure 7 A spiral process to deal with risk

The spiral process starts, naturally enough, when it is recognised that a particular organisational process can be improved or supported with the aid of a software system. The distance from the origin is intended to show how many resources have been used, such as the cumulative cost of a project.

There is no need for a complete solution to be produced by the end of the first iteration of the spiral. For example, the first iteration could focus on the question, 'Can we build an acceptable software system with the resources that can be brought to bear?' We might build a prototype as part of that first iteration.

After each iteration of the spiral, new risks come to light, and plans are made for the next iteration in order to resolve those risks. With successive progressions, you should reach a point where your review indicates that you have an acceptable solution – a software system that meets the needs of its users.

Such a risk-driven model is particularly helpful when developing large software systems or systems where the developers have little experience of the problem domain. In both cases there is a high risk of failure. For example, if you have never built a real-time system, you need to gain some understanding of the scheduled execution of tasks.

SAQ 9

- Why might a software development company specialise in a certain kind of customer, such as those in banking or health care?
- In which of the activities in Figure 4 would you expect to do your configuration management during a project?
- What are the additional risks affecting large projects?
- What is the effect of adding more people to a software development team?
- What important aspect is added to a development process with the introduction of risk management?

ANSWER.....

- (a) Through specialisation, a software development company can foster experience in a given area, whether it is banking, health care or any other field of interest. The developers in that company would have (or would hope to gain) sufficient knowledge to understand the problems raised by the users and (hopefully) be able to present solutions in a form that can be understood by those users. In addition, each company may develop and use a consistent development process that is appropriate to each set of customers.
- (b) Maintenance deals with change; configuration management is the discipline of managing and controlling change, and so you would expect maintenance to be where you would perform many of the configuration tasks. However, there is a role for configuration management during the development process in, for example, ensuring the consistency of models. Quality management is the activity in which you would perform these tasks.
- (c) The chances of failure increase as the size of a software project increases. Effective communication between the members of a large team becomes more difficult. There are also risks when developers have little or no experience of the problem domain.
- (d) The basic effect is that the number of communication channels between members of the development team will increase. The consequence is that team members will spend more time communicating with each other, rather than doing their allocated tasks, and the chance of introducing errors will be greater.
- (e) The most important additional aspect is the use of the reviewing, determination and evaluation steps that have been specified for a given process. These introduce feedback into the process to help ensure that the deliverables at each stage are leading, in a timely manner, towards the correct product.

3.4 Traceability

The need for a software system comes from a set of potential users, who may not be employed in the same company as the developers of the software system. Initially, developers must find out about the users' domain and express their understanding of the problem in a form that is appropriate for the proposed development process.

Early in the process, developers should produce a requirements specification document, which identifies what the proposed software system should do and the environment in which it must work. It is essential to relate what the users want to what the developers aim to provide. This implies that there is a need to record more than just the requirements. It must be possible to trace the history of each requirement from its origin in the problem domain, through the various intermediate activities, so that it is possible to reconstruct the significant events that led to the final, operational software system. The ability to trace the history of each requirement is known as **traceability**. Among other things, traceability is essential for dispute resolution (knowing what was agreed during a project), for seeing the effect, later in the project, of changing or deleting requirements, and for demonstrating that you have dealt with each requirement. The chosen development process will determine which events can be reconstructed, and hence determine the level of traceability. It is important to recognise that the amount of documentation should be commensurate with the desired level of traceability: neither too much nor too little!

On a given project, there may be a number of deliverable documents that record your activities from start to finish. You should treat your documents as part of that

explanation. Remember that the program code is a necessary, but not sufficient, deliverable that proves you have implemented a good software system. Documentation adds to your explanation, which we will expand upon as you study the rest of the course.

When you write down what you know about a problem, it helps to clarify your understanding of that problem. Furthermore, writing and drawing helps you to explore the problem and the potential solutions.

Keeping a *project notebook* is a disciplined approach to organising your thoughts and actions as a software developer. Your project notebook is a record of notes, thoughts, drawings, ideas and decisions (and the reasons for taking those decisions) as you work on a project. In its simplest form, a project notebook will be paper-based, but it could be recorded in files on a personal computer.

For it to be effective, you should keep your project notebook with you at all times, so that you can use it, save it and review it. There is no limit to what you can record in the notebook. For example, you can note down what your users say about their needs, make some preliminary sketches of models, or even record telephone numbers and URLs that may be useful while at work. Whatever you record, you should date-stamp your notes, and review them on a regular basis to ensure that your questions have been addressed and that you have extracted the information that is useful on a given project. You must keep accurate dates and times for the information recorded in your notebook for the following three reasons (all of which relate to traceability).

- 1 Your project notebook may be required as evidence in some enquiry or even in a court of law.
- 2 It helps you to review what you have done and how long you took to do some task.
- 3 It facilitates learning.

SAQ 10

- (a) Why is traceability important to the development of software?
- (b) How does documentation contribute to traceability within a development project?
- (c) Which activity or activities (shown in Figure 4) are most affected by poor documentation?
- (d) For what kind of software system might you minimise or even avoid any documentation?
- (e) Why is it important to review the contents of your project notebook?

ANSWER.....

- (a) Traceability is concerned with the reconstruction of significant events. In software development it should be possible to follow all the activities undertaken in response to a proposed change. In particular, you should be able to trace backwards from an implemented component or components, through their design, to a given requirement.
- (b) Within a development project, documentation records the progress from requirements to implementation (and beyond). It is possible to identify each requirement and follow the actions taken to implement a solution to that requirement. You might, quite simply, be required to show that you have implemented each requirement correctly.
- (c) All seven activities will be affected by poor documentation. The maintenance and quality management activities will be most affected, since they rely on the existence of traceability within the outputs of a development activity. (You saw this problem earlier when we looked at legacy systems.)

- (d) If a proposed software system is likely to have a very short lifetime and will be discarded after use, then it may be acceptable to minimise or avoid the task of documentation.
- (e) The act of reviewing your notes helps you to identify what worked and what did not, as long as the notebook records accurately what you did and when. A review helps you to trace the events that led to the decisions you made since your previous review.

3.5 Emergent approaches to software development

Iterative and incremental methods have been widely adopted in software development. Nowadays, high competitiveness, reduced time-to-market and pressure to develop flexible enterprise software, together with the rapid change of technology, have led to the emergence of new approaches to building, deploying and maintaining software. At the time of writing, several new approaches to software development have been established that may become significant during the lifetime of this course. These include Model Driven Architecture (MDA) (see <http://www.omg.org/mda/>) and agile development (see <http://agilemanifesto.org>).

Model Driven Architecture

MDA defines an approach whereby business-oriented decisions and specifications can be separated from implementation and platform (technology) decisions. The idea is to distinguish between platform-independent models (PIMs), specified using UML, and platform-specific models (PSMs), which carry relevant information for the generation of platform-specific code. Standard mappings between PIMs and PSMs should allow tools to be developed to automate some of the implementation. This approach is based on a strong emphasis on:

- ▶ models specified in a well defined notation;
- ▶ transformations between models;
- ▶ formal underpinning of models and transformations;
- ▶ industry-adopted standards and tools.

Agile development

Agile development is an umbrella term used to describe a variety of (agile) methods that encourage continual realignment of development goals with the needs and expectations of the customer. It represents a compromise between no process and too much process: a lighter-weight, faster and nimbler software development process can adapt to the inevitable changes in customer requirements.

Leading proponents of the agile approach have proposed the following manifesto:

Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right,
we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

© 2001, the above authors.

This declaration may be freely copied in any form, but only in its entirety through this notice.

Source: www.agilemanifesto.org [accessed 26 June 2007]

eXtreme Programming (XP) (Beck, 2000) is one of the best-known agile methods; it is a very lightweight method, based on intensive testing and incremental development. It defines a series of practices such as small releases, simple design, testing, programming in pairs, collective ownership, continuous integration, a 40-hour week, an on-site customer, and coding standards.



Exercise 2

What are the main problems associated with the development of software?

Solution

Here is our list of problems that affect software development (you may have identified others):

- ▶ difficulty in capturing all the user requirements;
- ▶ difficulties in managing and developing large and complex systems;
- ▶ difficulties in meeting users' requirements and avoiding dissatisfaction with the completed software;
- ▶ continual changes in user requirements (often referred to as 'requirements creep');
- ▶ difficulties in maintaining software;
- ▶ the presence of errors introduced during development;
- ▶ a changing world leading to changes in the needs of the users and decreasing the usefulness of the software;
- ▶ apparent low productivity of those developing software, and difficulties in measuring that productivity;
- ▶ maintaining momentum and avoiding getting 'stuck', especially when the development process hides the goal of delivering a good software system;
- ▶ ensuring the quality of a software system.

Exercise 3

- (a) How do you think a large-scale project differs from a small-scale project in terms of the management of the development team?
- (b) What steps might be taken to tackle the problems that arise?

Solution

- (a) As the size of a project increases, so does the number of people involved, and consequently the complexity of the interactions involved. For example, in a two-person project there is only one pairwise path of communication; with three people there are three pairwise paths; with four there are six pairwise paths; and so on. As with programs, the greater the complexity of interaction, the greater the chance of errors occurring.
- (b) One of the functions of software project management is to reduce this complexity. One way to achieve this aim is to modularise project teams by splitting them up into a number of groups. Each group performs a specific task and communicates with the other groups through one person (for example, the head of the group).



Exercise 4

Life-cycle models sometimes show only the software-creation activities and exclude software maintenance. What problems are associated with the use of such models?

Solution

If maintenance is not mentioned, it is likely that it will not be considered to be an integral part of the overall software life cycle. This may result in a design that does not take the needs of maintenance into account, and hence a product that is difficult and expensive to change. This is especially likely if time and/or budget are already tight.



Exercise 5

What kinds of software system would be best suited to the incremental model?

Solution

Software systems where the problem falls naturally into partitions, each of which represents a fairly self-contained unit that could be developed and delivered on its own to provide the users with a useful chunk of functionality, would be well suited to this model. The incremental approach allows several design issues to be tackled simultaneously and enables the system to be implemented in relatively small steps in order to contain the complexity. Of course, this assumes that the design can be partitioned.



Exercise 6

Figure 7 shows a spiral process for software development. Into which quadrant of that figure does each of the seven technical activities fit?

Solution

The analysis, design, implementation and testing activities all fit into the 'develop and verify a (partial) solution or product' segment. The analysis activity also overlaps the evaluation quadrant where objectives and risks are analysed. The testing activity overlaps the 'review and plan' quadrant. The maintenance, project management and



quality management activities (including configuration management) operate in all four quadrants.



Exercise 7

It has been said that ‘the maintenance activity begins with the first deliverable of a development project’. What does this imply about the maintenance activity?

Solution

This question refers to the fact that there could be deliverables, such as requirements documents, that exist prior to any software being built. The maintenance activity allows a software system to evolve. It is the activity that responds to changes. For example, you might detect errors or inconsistencies in a document that lists the important terms in your users’ domain; or you might have to respond to changes in the environment, which may be due to new regulations or market forces.

3.6 Summary of section

This section considered how you might approach the development of a good software system as follows.

- 1 Large software projects are prone to problems because of their size and complexity
- 2 Software is inherently easy to change, and this makes it easy to introduce new errors into it (‘break’ it).
- 3 Software development is similar to engineering when it involves a defined process with clear activities, each of which has one or more products that can be tested against the users’ requirements.
- 4 A basic process for the development of software consists of methods that describe the technical activities and procedures. It helps to coordinate and control the use of those methods and any tools that support them.
- 5 Project life-cycles can be sequential, iterative, incremental or some combination of these.
- 6 A disciplined approach to software development implies that you must make some effort to record your activities from both a personal view and a project view. It means that documentation is an important task for all those who work on a project. In particular, it must be sufficient for the level of traceability required.

4

Modelling

In this section we will consider the importance of models as part of software development. In particular, we will consider the role of a standard modelling language.

4.1 From methods to modelling

You have already seen how a particular form of modelling, namely prototyping, can contribute to the development of a software system. Much of this course is concerned with modelling in general and the use of diagrams to represent models in particular.

Modelling is a way of thinking about things and ideas in the 'real world'. A model is an abstract representation of a situation in reality, or of a system from a particular point of view. In effect a model is a way of expressing a particular view of an identifiable system of some kind.

In terms of the development of software systems, **models** are:

- ▶ a way of understanding the problems involved;
- ▶ an aid to communication among those involved, especially between the developer and the user, as part of some deliverable;
- ▶ a component of the methods used in development activities such as analysis and design.

A good model is an abstraction, which allows those involved to concentrate on the essentials of a (complex) problem by keeping out non-essential details. In Section 2, you saw that there is a limit to how much a person can understand at any one time. So we build models to help us in activities such as the development of software systems. For example, developers build different models at different stages during the development process in order to confirm that the eventual software system will meet the users' requirements.

Models contain a representation of the significant states, objects and events within a software system. In one respect, models are an idealisation, because they are less complicated than reality, and so they are easier to use for software development. The benefit arises from the fact that only the relevant properties of the outside world are represented. For example, a road map is a model of a particular part of the earth's surface. We do not show things like birds' nests as they are not relevant to the map's purpose. We use a road map to plan our journeys from one place to another, and so the map should contain only those aspects of the real world that serve the purpose of planning journeys.

When we model, we are trying to show or reveal what something is like. Models can do more than this: they can help us explain the past and the present (the problem), and they can help us predict and control the future (the solution – a software system). However, remember that the model and the real world are alike in some ways, but different in others. For example, road maps are helpful because they represent distances between (and relative positions of) places, along with routes between them. They use the relevant properties of the real thing with just a change in scale; one centimetre on the road map, for instance, may be equivalent to one kilometre on the ground. They may be unhelpful if they show only major roads.

We can also use one property in a model to represent another in the real world. In the case of the road map, we can use different colours to represent different classes or types of road. Such a road map should have a key or legend so that those who read the map can understand what the different coloured lines are intended to represent. On a road map, a blue line might represent a motorway, and a yellow line might represent a narrow track.

Models of a problem situation are only an approximate representation of that situation. The real-world situation is normally complex – so much so that an exact representation is likely to be impossible to achieve. Therefore, the problem confronting a developer is to find some way of achieving an acceptable balance between completeness and manageability. In a software development project, there will be a number of practical considerations that result in some compromise in completeness.

If the constructed model is so complex that the developer (or other team members) cannot use it, then it is of little or no value. So the model must simplify the reality a good deal. However, the developer should make explicit all simplifying assumptions for a given model, as well as their consequences. At some point in a development project, any of these assumptions may need to be justified. This is another aspect of the level of traceability within a development project, which we will return to later in this section when we consider the recording of models.

Models are subject to change. At the very least, they require some form of periodic testing so that a model can maintain its correspondence with reality. As towns and cities expand and contract, a road map must be changed to reflect the new situation. In the worst case, a change in scope necessitates a whole new model. For example, if there were a need to reflect the current status of roads and the traffic density on them, a simple road map would be inadequate.

A standard notation

In developing a software system, a developer will not just use a single 'catch all' model; a set of related models is more likely. It would be preferable to have a consistent way of representing each of the different models for a given software system. A **modelling language** allows the developer to make useful connections between those different models. In software development, a modelling language is often based on diagrams and their construction, meaning and use. There are two sets of rules within a diagram-based modelling language:

- ▶ one that determines what diagrams exist and what symbols are used on each one – its *syntax*;
- ▶ one that determines what the diagrams and symbols mean – its *semantics*.

What should you look for when choosing a modelling language? On any given development project there are decisions to be made about every model. You should favour a modelling language that:

- ▶ allows you to express the many facets of the subject of your model;
- ▶ helps you to resolve misunderstandings rather than introducing new ones;
- ▶ is easy to learn and use – you want to make progress quickly;
- ▶ is supported by tools that allow you to use your modelling skills rather than your drawing skills;
- ▶ is used widely and is accepted within the industry.

It is advantageous if a modelling language is widely used. If you leave one team of developers and join another that uses the same modelling language, then you need only learn about the new problem situation. The advantage is amplified if you move to a new team in a new company! Naturally, the team that you join will expect you to be

more productive than you would be if you had to learn a completely new modelling language.

In the same way, there is a greater chance that a module developed for one software system can become a component in another system if a standard modelling language was used in the design and implementation of that module. For example, suppose you were trying to decide which electric motor to include in a new vacuum cleaner. Clearly, you would need to begin by reading all the descriptions. The chances of choosing an appropriate electric motor are increased if you can understand the descriptions of the motor's specification, to be confident that it will meet your requirements.

The development of software systems according to object-oriented principles has gathered pace in the last decade or so. But there were a number of different modelling languages in use while opinions varied about what it meant to 'develop object-oriented software systems'. **Unified Modeling Language (UML)** has emerged as a clear leader in the field. Its success is partly due to its separation from any particular method. It is available for anyone to include in their own method for software development. Although a detailed history of UML is beyond the scope of this course, you should know that it was derived from three of the most popular methods that were used in the earlier projects that adopted object-oriented principles. The Object Management Group (OMG), an industry consortium for modelling and integration standards, has adopted UML and is the main body responsible for its development.

If you want to know more, you can go to the OMG's website at <http://www.omg.org>.

UML is a standard modelling language and is the one we have chosen for this course. It is intended to be a general-purpose language for software development. It is not meant to be a complete language for modelling all aspects of all systems.

UML is predominantly diagrammatic, but does allow developers to add text in appropriate places.

SAQ 11

- (a) What is a model?
- (b) What is a 'good model'?
- (c) What are the two kinds of rule that govern the use of a modelling language?
- (d) Does a modelling language need to be associated with a particular development process?
- (e) What are the required characteristics of a standard modelling language?
- (f) How does a standard modelling language contribute to software development?

ANSWER.....

- (a) A model, in terms of software development, is an abstract representation of a specification, a design or a system, from a particular point of view. In general, it is a simplification of reality, created in order to understand an aspect or viewpoint of the system in question.
- (b) A 'good model' is an abstraction that allows those involved to concentrate on the essentials of a complex problem by excluding non-essential details while capturing those of interest.
- (c) A modelling language is normally diagrammatic, although it can be textual. In common with natural language, there are two distinct kinds of rule:
 - 1 those that determine whether or not a diagram is legal – the syntax of a diagram;
 - 2 those that define what a legal diagram means – the semantics of a diagram.

- (d) No, although a particular development process is likely to define its own modelling notation.
- (e) When choosing a modelling language, it should be:
 - ▶ sufficiently expressive;
 - ▶ easy enough to use;
 - ▶ unambiguous;
 - ▶ widely used;
 - ▶ supported by suitable tools.
- (f) A standard modelling language helps when new people join a project: a common modelling language reduces the time needed to enable them to become productive team members. Also, when a modelling language is widely used, it is likely that project components will have been constructed using that language. This makes the software easier and cheaper to maintain.

4.2 Models illustrate points of view

To understand the architecture of a software system, you will need a number of complementary and related views. If the main influence is the users, then a view that expresses their requirements is essential. This is quite normal. Every development method you will encounter will encompass one or more views of the software systems within its scope.

During software development, you will be interested in different aspects or *views* of the problem, and its solution at different times. It follows that you will construct different models to suit those aspects. As mentioned above, it's unrealistic to expect to put everything into just one model. Too much detail in a model can only be a distraction.

When it comes to the development of an object-oriented software system, there are two distinct kinds of model:

- ▶ **static models**, which describe the objects in a (software) system and their relationships to other objects;
- ▶ **dynamic models**, which describe the behaviour of a (software) system over time.

In practice, such a simple partition of a software system is not enough. For example, in a distributed system a developer must also consider the potential location of the modules (or classes). So the overall process of software development takes a number of different views into account. A software architecture is a way of identifying the differing views.

A software architecture is not likely to appear after a single short burst of activity. It is more likely to arise from a well structured process that involves a series of successive refinements. There is no strict requirement to use a modelling language to produce an architecture for a system, but there are benefits from using one.

Example 11 illustrates a set of views of a software architecture that has been adopted in the Rational Unified Process, which uses UML as its standard notation. In practice, the developers are likely to produce a software architecture for the software system during certain project activities such as analysis or design. Each view can be thought of as a model that expresses a particular aspect of the overall system; each one is an abstraction. The views interact with each other; this will become evident as you continue your study of the course.

Example 11

The Rational Unified Process, which has been developed by the same people who originally specified UML, has five views of a software architecture, as follows.

- ▶ **The use case view.** This contains the basic scenarios that describe the users and the tasks that they need to perform with the aid of a software system. These scenarios are partitioned into use cases, which we will describe in *Unit 3*.
- ▶ **The logical view.** This is concerned with the functional requirements of the software system. What should the software do for its intended users? Typically, this involves the construction of one or more class diagrams, which we will describe in *Unit 5*.
- ▶ **The implementation view.** This is concerned with the organisation of the modules that comprise a software system. Typically, it addresses the management of source code, data files and executables.
- ▶ **The deployment view.** This is concerned with the relationship between the various executables (and other run-time components) and their intended computer systems. We deal with this aspect in *Unit 9*.
- ▶ **The process view.** This is concerned with aspects of concurrency. What are the processes and threads? How do they interact? It deals with such things as response time, deadlock and fault tolerance. This is not dealt with in this course, but is covered in M362.

Figure 8 shows how the central use case view relates to the other four views.

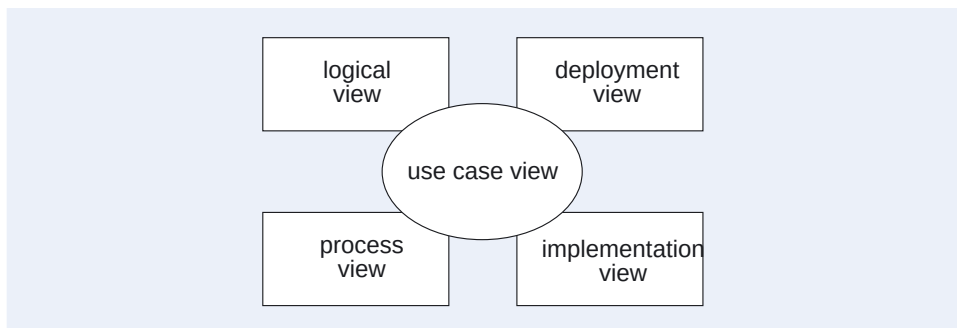


Figure 8 Five views of a software architecture

Views in UML

You will be introduced to the terms used in UML as you study later units. In UML, a model is treated as a description of a system at some chosen level of detail from one particular view. As development proceeds, each model undergoes change; there are successive refinements until a model is deemed complete with respect to the chosen view. Having identified the essential concepts in the problem domain as classes, for example, the developer will add attributes and operations as part of the activity of requirements specification. During design, the developer adds more detail and refinements until the classes can be implemented in the chosen programming language.

In a given model there will be a number of distinct elements, such as classes, use cases, associations and so on. Each one will have an individual representation in a diagram, for example, boxes for classes, ovals for use cases and lines for associations. In UML, an **element** is an atomic constituent of a model. Classes and

UML assumes that a software system is modelled as 'a collection of discrete objects that interact to perform work that ultimately benefits an outside user'.

associations are the model elements to be found in class diagrams, for example. The boxes in a class diagram are presentational elements. A **diagram** is a graphical presentation of a collection of model elements. (Beware! Some elements, such as arrows, have different meanings in different diagrams.)

SAQ 12

- (a) Should we try to capture everything about a design in a single diagram?
- (b) What is the difference between a static and a dynamic model?
- (c) The diagrams within a single model must be consistent. Does the same apply to all the models used in a given development project?

ANSWER.....

- (a) No, because we will be interested in different aspects of a design at different times, and different models of our design will be built to reflect our interpretations of users' needs.
 - (b) A static model describes the elements of the system and their relationships to other elements. A dynamic model describes the behaviour of a system over a period of time.
 - (c) Yes; the whole set of diagrams should contain the different aspects of a single software system. So they should not contradict one another. For example, there must be some consistency-checking between the static and dynamic models; this can be automated by a suitable tool.
-



Exercise 8

Suppose you and a colleague used UML on a particular development project. What would be the consequences of introducing symbols or notations that were not defined in UML?

Solution

Such an extension would incur costs as well as offering potential benefits. The costs include the need to communicate the syntax and semantics of your new symbols or notation to other developers.



Exercise 9

In the past, there have been many attempts to standardise modelling notations and languages. How might the choice of modelling language affect the maintenance of a software system?

Solution

Every time a new standard is defined, it allows new software systems to be built using that standard. It can only help with existing software retrospectively. For example, a developer might use the new standard to express a proposed change to the software.

There is no guarantee that an existing software system used any modelling language at all, let alone any agreed standard for one. Even if a standard was used, new staff arriving on the team for the purpose of maintenance might not know about it. Each variety of standard would affect the organisation of a component library. For example, if components using a particular standard were to be put in a library, the descriptions would be likely to require some maintenance as that standard changes over time.

4.3 Summary of section

As you become more familiar with the activity of modelling, it should become apparent that there is considerable flexibility in the construction of models. There is no guarantee that different developers, when confronted by the same problem, will produce identical models, although there should be a great deal of similarity in most cases.

This section considered the role of modelling in the development of a good software system as follows.

- 1 A standard modelling language helps those involved in software development projects to communicate effectively. If the standard is widely used, it will reduce the time taken for developers to become familiar with a new project.
- 2 The Unified Modeling Language (UML) is a useful standard because it is easy to use, sufficiently expressive and widely used. There are also a variety of tools that support UML.
- 3 A model is an abstract representation of a concept, a specification, a design or a system from a particular point of view of either the user or the developer. In general, it is a simplification that is used to understand an aspect of the system in question.
- 4 During software development, there may be many models made from a variety of viewpoints. They must not contradict each other; that is, they must be consistent.

5

Activities and artefacts

In this section, we identify the core activities we will use in this course to develop software. This will give you an initial understanding of how the activities you will learn about contribute to the overall development process. As noted earlier, different ways of subdividing the development process and different terminology may be used elsewhere.

In later units we will be referring to the people who carry out the related roles. We will use the term **developer** for anybody involved in the development of the system. We will use the term **analyst** or **requirements engineer** for anybody involved in domain modelling or requirements engineering. Note, however, that these and other names really do refer to roles rather than people. Especially on small projects or in small companies the requirements engineer may well be the same developer who designs and tests the system etc.

Our core activities are as follows.

- ▶ **Domain modelling.** Modelling what already exists.
- ▶ **Requirements engineering.** Specifying what the system must do.
- ▶ **Further analysis.** Adding detail to the system specification
- ▶ **Design.** Showing how the system will meet its specification.
- ▶ **Implementation.** Producing code that will meet the user requirements.
- ▶ **Testing.** Ensuring that software does meet its requirements
- ▶ **Deployment.** Configuring the code to give a runnable system.

It therefore makes sense to talk of models corresponding to each of these activities. So we have a domain model, a requirements model etc. However, these models generally need to cover a number of views of the system. For example, in the last section we talked about static and dynamic models. A domain or a design model, for example, will usually include both of these. So a design model typically consists of a number of more specific models. There will be other documents that accompany the models, and so we will use the term *artefact* to refer both to models and other documents. The artefacts from the activities earlier in the list above feed into the activities that follow.

5.1 Domain modelling

Our starting point will be to document and model the structure and processes of the organisation's business. Starting from some description of the problem (the **initial problem statement**), you will learn how to identify elements of the real-world problem and their properties, and build corresponding structural and dynamic models. The emphasis at this stage is on understanding the current situation.

The *dynamic model* provides descriptions of **business processes** and behavioural aspects of the domain. You will meet dynamic models such as activity diagrams and state-machine diagrams. **Business rules** are used to express constraints on the dynamic model. For example, given a dynamic model describing aspects of a library, there might be a business rule determining the limit on the number of books an individual member can borrow. A **structural domain model** (concept diagram) describes the significant concepts in the domain, and how they are related.

Also produced is a **glossary** of relevant terms and definitions. The glossary is a textual document that grows throughout the duration of the project.

Some artefacts involved in domain modelling are illustrated in Figure 9.

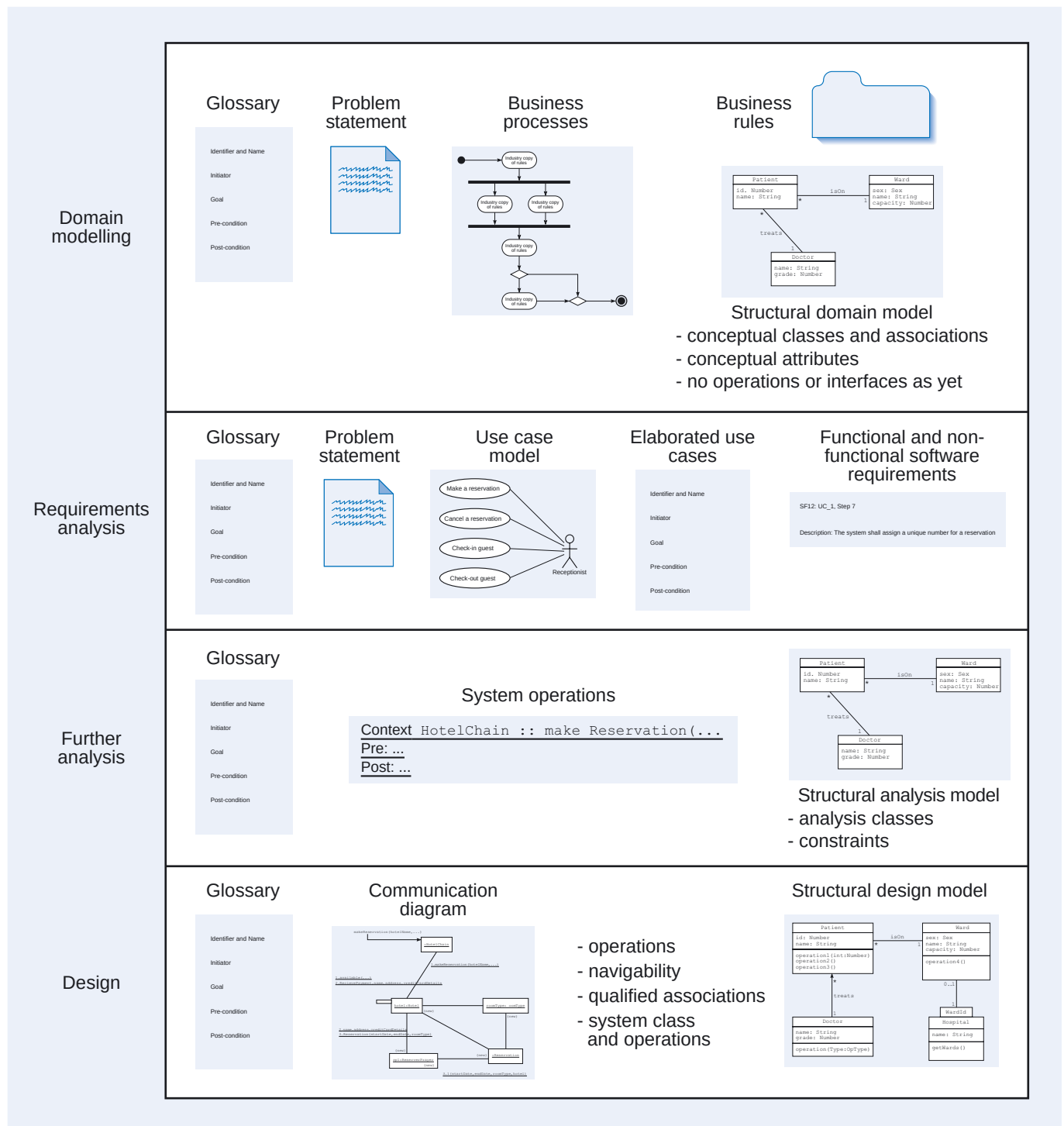


Figure 9 Some development artefacts

SAQ 13

- (a) What is the purpose of domain modelling?
 (b) What is the role of each of the artefacts produced during domain modelling?

ANSWER.....

- (a) Domain modelling is concerned with gaining an understanding of the environment in which any system that is designed must operate.
 (b) During domain modelling, we produce the following artefacts.

Dynamic model: a description of the business processes and behaviour of the domain.

Business rules: constraints on the way the above model operates.

Glossary: definitions of relevant terms.

Structural domain model: an initial structural model representing the concepts relevant to the domain.

5.2 Requirements engineering

The next step is to gather and document the requirements for our system. Requirements are the expression of the things the system must do or the qualities the system must have in order to meet the stakeholders' needs. Starting from some description of the problem, you will learn how to systematically record requirements information. *Unit 2* discusses requirements, how they are recorded, and the properties we expect them to have. In particular, you will be shown how to use the **Volere template**, which provides a disciplined way of recording requirements. We model requirements at various levels of detail, using **use case diagrams**, use cases elaborated as text and detailed **software requirements** stating what the system should do.

Some artefacts involved in requirements engineering are illustrated in Figure 9.

SAQ 14

What is the purpose of requirements engineering?

ANSWER.....

Requirements engineering is concerned with establishing what a software system must do.

5.3 Further analysis

During *further analysis* we complement the work on requirements engineering by providing a more detailed specification of what the system must do. We need to ensure that we have the right system operations in our system to support each of the software requirements associated with it. As well as identifying and specifying these system operations, we elaborate on the structural model by specifying further constraints on it.

Some artefacts involved in further analysis are illustrated in Figure 9.

5.4 Design

During design, our goal will be to decide how the expected functionality is to be allocated to each part of the system. We need to make choices about which classes the system operations should be allocated to. These decisions can be explored and documented using further dynamic models. An example of a UML diagram for doing this is the **communication diagram** that shows a set of classes and the messages between them.

During design, then, both structural and dynamic models are elaborated. In fact, we will see that dynamic models are used for both external and internal behaviour of objects.

Some artefacts involved in design are illustrated in Figure 9.

SAQ 15

- (a) What is the purpose of design?
- (b) What is the role of each of the artefacts produced during design?

ANSWER.....

- (a) Design is concerned with making decisions concerning how a system will meet its specification.
- (b) During design we produce the following artefacts:
 - Structural model: This is an updated version of the one produced during domain modelling but with its operations specified.
 - Behavioural models: These show how objects in the system will interact and behave internally, and also how functionality will be distributed across the system.

5.5 Implementation

The *implementation* model is a description of the assembled components that comprise the working version of the software. It will describe how classes are packaged together, and show some of the relationships between such packages of classes. These relationships are modelled using a component diagram.

5.6 Testing

The implementation needs to be tested against its requirements to ensure that it meets them. Tests will be drawn up based on the requirements, and held in a test document. Notice therefore that tests can be drawn up as soon as we know the requirements. In fact there is a trend towards test-driven design. In this approach, the tests for a system are drawn up before design and automated testing procedures are set up. An initially empty implementation can be run against the tests, and the subsequent failure can be used to drive design. As time goes on, this process is repeated as the implementation is built up, until no tests fail.

5.7 Deployment models

Many significant computer systems will consist of a variety of software components located on a number of machines communicating via a variety of hardware and software mechanisms. A deployment model records how various components are to be mapped onto different machines, and how they will communicate. This can be represented using a deployment diagram.

SAQ 16

When is the earliest point that you might need to produce a deployment model in a development project?

ANSWER.....

The users of a software system may be dispersed throughout the organisation, and you should expect that their requirements would reflect this. Your conceptual model should avoid this aspect of the requirements, but some consideration of the users' diverse locations would be expected. Thus you should expect to produce some form of deployment model very early in the project, as a separate design activity, after the separation of the logical and physical requirements during analysis. What is in the model will depend on the nature of the requirements. For example, you might be asked to develop software to suit an existing hardware configuration, which is a physical constraint.

6

CASE tools

Computer-aided software engineering (CASE) tools were created to support the professional software engineer in the complex task of developing large information systems, and to improve their productivity. Below we consider the benefits and costs of their use, and the range of tools available.

6.1 Benefits and costs

Many benefits may accrue from the use of CASE tools. From a developer's viewpoint, they provide support for modelling aspects of the system using a variety of notations and techniques, from diagrams to mathematics and text, producing prototype code, and even verifying the correctness of the system design. They also automate the sometimes tedious process of writing system documentation and keeping it up to date. They often allow the creation and management of a central repository of documents and other artefacts. This is useful both for communication within a development team, and for project management and decision tracking. Importantly, CASE tools improve the quality of the development processes by supporting, and to a large extent enforcing, a standard methodology and sound design principles.

Developers use a CASE tool in order to:

- ▶ coordinate access to the different models of the problem and its software solution;
- ▶ help with the collaboration between team members;
- ▶ share the same concepts in different models;
- ▶ store multiple versions of the different models;
- ▶ identify differences between versions;
- ▶ check the consistency and integrity of diagrams and models;
- ▶ generate project reports and other documents;
- ▶ generate basic structures and program code, sometimes called **forward engineering**;
- ▶ generate models from an existing implementation, sometimes called **reverse engineering**.

Despite the potential benefits of CASE tools, there are often obstacles to their adoption. The introduction of a CASE tool within an organisation comes at a high cost: an upfront investment is required to acquire the technology and to train personnel accordingly, while the benefits of using the tool manifest themselves only in the long term. CASE tools are only effective if standard methodologies and processes are adopted across the organisation; the definition of standard procedures and practices also has to be established at the beginning. At times the obstacles are cultural: there may be some resistance from analysts and developers who perceive the rigidity of an enforced methodology as a threat to their autonomy and creativity. The initial learning curve is quite steep, and developers may not consider the effort worthwhile in terms of improvement in productivity and quality. Another major drawback is that the available tools tend to be narrow in their focus, and concentrate on a small subset of activities within the development process. For instance, most tools support modelling, while hardly any provide facilities for communication between customers and developers. This may result in a variety of tools being used within a process, with all the difficulties arising from integration and interchange of information.

In this course, we have chosen to use two tools: a very lightweight modelling tool and a programming tool. We have judged from past experience that the learning curve for a fully fledged CASE approach is not justified in this course, and that it might detract from the learning of more fundamental material.



Exercise 10

Which factors may contribute to the successful adoption of CASE tools within an organisation?

Solution

Across the organisation, there should be established methods and procedures that can be supported by the tools.

Adequate investment should be put into training managers and developers.

Deployment should occur over a long period of time, as the benefits of CASE tools are not short term.

Clear procedures should be established for the use of different tools within development, and standards should be followed for information exchange among tools.

To overcome some of the deficiencies of current tools, they should be integrated within practices that support project management and customer–developer communication.

6.2 A taxonomy of tools

A great variety of tools are available today, supporting many different functions within software development. Table 2 gives a brief taxonomy of tools based on their function; the list is not meant to be exhaustive. Also, many current CASE systems integrate many such functions within the same application environment.

Table 2 Brief taxonomy of CASE tools

Tool	Function
Process management tools	Capture and model development processes
Project management tools	Plan and schedule projects, and track progress
Risk analysis tools	Record, categorise and analyse risks
Requirements management tools	Capture, analyse and trace requirements throughout development
Metrics tools	Capture specific software metrics and provide overall measures of quality
Modelling tools	Support modelling within analysis and design activities
Programming tools	Support code development
Interface design tools	Support the design of graphical user interfaces (GUI)
Test management tools	Manage and coordinate testing

Particularly relevant to this course are UML modelling tools, which support the creation of models based on UML techniques. Many such tools are available, from simple diagrammatic tools to full-blown integrated environments with project management, document-repository and code-generation capabilities. You will have an opportunity to explore the capability of one such tool during the course.

6.3 Summary of section

In this section you have learnt how a modelling tool organises and records the diagrams and descriptions that represent the different aspects of a software system during its development.

7

Summary

In this unit you have extended your knowledge of the important ideas in software development. We investigated the characteristics of a good software system, and considered what a development process would need to include to build such software. You saw that there is no single development process to suit the variety of users' requirements. However, there must be a disciplined approach to software development, especially in the case of large projects.

We introduced the notion that it is good practice to split a project into smaller, more manageable activities. When developing good software systems, you should focus on the users' needs and, wherever possible, make use of replaceable and reusable modules – components. The overall software architecture should be constructed around the users' requirements.

We then introduced the role of modelling in the development of software. In particular, the concepts of object orientation allow us to represent users' requirements in a way that reflects our natural tendency to view the world around us in terms of objects. The way we relate the various activities of software development and associated artefacts (including models) was then described. Finally, we discussed CASE tools.

Now that you have more knowledge of the software development process, you may wish to read Section 2 again.

LEARNING OUTCOMES

On completion of this unit you should be able to:

- ▶ describe the essential characteristics of a good software system;
- ▶ identify, using examples, the connections between the characteristics of a good software system;
- ▶ explain why a software system can be out of date even as it is being developed;
- ▶ explain the importance of modularisation;
- ▶ suggest reasons why large software projects are prone to problems, and give examples to support your case;
- ▶ give examples to support the notion that software development can be considered as an engineering discipline;
- ▶ describe the elements of a basic software development process;
- ▶ illustrate the variety of different life cycles by using examples;
- ▶ explain the role of documentation in a software development project;
- ▶ explain why a standard modelling language is beneficial to a development process;
- ▶ explain the benefits of the Unified Modeling Language (UML) as a standard notation for modelling;
- ▶ identify the different kinds of model used in the development of software;
- ▶ describe the relationship between models, viewpoints and software development;
- ▶ understand the relationship between activities and artefacts used in this course;
- ▶ understand the role of CASE tools and the issues involved in their adoption.

References

- Bass, L., Clements, P. and Kazman, R. (2003) *Software Architecture in Practice*, 2nd edn, Boston, Addison-Wesley.
- Beck, K. (2000) *eXtreme Programming Explained: Embrace Change*, Boston, Addison-Wesley.
- Cheesman, J., Daniels, J. (2000) *UML Components: A Simple Process for Specifying Component-Based Software*, Boston, Addison-Wesley.
- Page, D., Williams, P. and Boyd, D. (1993) *Report of the Inquiry into the London Ambulance Service*, The Communications Directorate, South West Thames Regional Health Authority.
- Pressman, R.S. (2001) *Software Engineering: A Practitioner's Approach*, (5th edn.), McGraw-Hill.

Index

A

abstraction 17

analysis 13, 24, 28, 39

architectures 19

B

business processes 46, 48

business rules 46, 48

C

circular dependency 15

cohesion 17

communication diagram 49

component 19

computer-aided software
engineering (CASE) tools 5, 51

configuration management 30

context dependencies 16

contracts 16

coupling 15

customers 24

D

decomposition 12
 into layers 16

deliverables 23

design 9, 13, 24, 28, 39, 43, 46,
49

development
 artefacts 47
 method 28

diagrams 44

domain modelling 46, 48

domains 7

dynamic models 42

E

elements 43

encapsulation 16

engineering 5

 forward 51

 reverse 51

environments 7

F

forward engineering 51

further analysis 48

G

glossary 47

H

heroic programming 13

I

implementation 13, 24, 28

incremental development 27

increments 27

initial problem statement 46

integration 24

interfaces 16

iteration 26

L

legacy systems 10

life cycles 23

M

maintenance 24

malleability 8

methods 23

model elements 44

modelling 39

modelling languages 40

models 39

modularisation 14

modules 12, 14

O

Object Management Group
(OMG) 41

P

partitions 14

presentational elements 44

problem domains 13

procedures 23

processes 6

project management 22

project notebooks 34

projections 14

prototypes 26

Q

quality management 25

R

Rational Unified Process (RUP) 43

requirements 48
 elicitation 24
 engineering 46
 specification 24

reuse 16

reverse engineering 51

risk assessment 31

S

software
 affordability 9
 architecture 19, 42
 availability 9
 complexity 8
 flexibility 9
 maintainability 10
 quality 5
 reliability 9
 requirements 20, 48
 size 8
 usability 9
 usefulness 9

software systems 8

- spiral process 32
 - standard modelling languages 41
 - standard notation 40
 - static models 42
 - structural domain model 46
 - system architecture 19, 42
 - system boundaries 7
 - systems 6
- T**
- test-driven design 49
 - testing 24, 28, 46
 - tools 23
 - traceability 33
- U**
- UML diagram 49
 - Unified Modeling Language (UML) 41
- use case diagrams 48**
- users 24**
- V**
- views (of a software problem) 42
 - Volere template 48
- W**
- waterfall model 25

